

Complete Technical Study Guide

Multi-Asset Next-Day Return Prediction — Bitcoin, Gold, Silver

Final Year Project (BSCS), University of Lahore — team “teamlocalhost”
Repository name: crypto-forex-prediction-system

Prepared for the supervisor presentation / viva. Written **from the actual code, data files, result files and a fresh test run** of the repository (not from memory or from the README alone). Where the code differs from what one might assume, this guide documents the code.

How to use this guide. Chapters 1–9 explain the problem, data, target, features and split. Chapters 10–19 explain models, training, evaluation, ensemble and the prediction flow. Chapters 20–23 are the VS Code map (every file, every important function). Chapters 31–34 and 40 are the viva material (50+ Q&A, 2-minute and 5-minute scripts, slide content, one-page cheat sheet). Chapter 39 is the honest assessment — read it before the viva.

One-sentence truth to keep in mind throughout: the project builds a rigorous, leakage-audited pipeline and honestly finds that **none of the models beats the “tomorrow = today” baseline** for next-day returns of Bitcoin, Gold or Silver; the one part of the output with demonstrable skill is the size of the uncertainty band, not the direction of the price.

Verification record (what was actually executed while writing this guide): full test suite (43 passed), a look-ahead truncation test on the real data, scaler/split checks, a headless run of all five dashboard tabs for two assets (no exceptions), and the real prediction path. See Appendix A.

Contents

1. Executive Summary	6
1.1 In very simple English (what you would say to an examiner)	6
1.2 The questions an examiner will ask first	6
1.3 Technical summary	6
2. Project Objectives	7
2.1 Primary objectives (supported by the implementation)	7
2.2 Secondary objectives	7
2.3 Why each design choice exists	7
3. Problem Statement	8
3.1 Why prices are hard to predict (evidence from our own data)	8
3.2 Limitations of the approach	8
4. Complete System Workflow	9
4.1 Stage-by-stage table	9
5. Dataset — Complete Details	10
5.1 Source	10
5.2 The three assets	10
5.3 External series row counts (data/raw/)	10
5.4 Raw columns (every column of the OHLCV files)	10
5.5 Data-quality handling (what the code really does)	11
5.6 Scaling and normalisation	11
6. Target Variable	13
6.1 What is predicted	13
6.2 Where and how it is created	13
6.3 Why this target	13
7. Feature Engineering — every feature of the final models	14
7.1 Which features each asset uses	14
7.2 Feature dictionary	14
7.3 The close-time rule (the most important design idea in the project)	15
7.4 Quirks you should know (verified in code)	15
7.5 Features that are NOT model inputs	15
8. Data Leakage Audit	17
8.1 Residual risks (honest list)	17
9. Train / Validation / Test Split	18
9.1 The frozen split	18
9.2 Strategy used: chronological + expanding-window walk-forward	18
9.3 Why chronological splitting is essential	18
9.4 Limitations of this validation design	18
10. Machine-Learning Models	20
10.1 Naive (random walk) — the baseline	20
10.2 Naive-Mean and ARIMA — the other baselines	20
10.3 Ridge regression	20
10.4 Random Forest	21

10.5 LightGBM	21
10.6 CatBoost	21
10.7 GRU and LSTM (recurrent neural networks)	22
10.8 The stacked ensemble (experiment, not served)	22
11. Model-by-Model Mathematical Explanation	23
11.1 Common set-up	23
11.2 Ridge	23
11.3 Random Forest	23
11.4 LightGBM and CatBoost (gradient boosting)	23
11.5 GRU and LSTM	23
11.6 Evaluation mathematics used later	23
12. Hyper-parameters	24
12.1 How they were chosen	24
12.2 Final values (results/tuning/best_params.json) and learned stopping points	24
13. Model Training Pipeline	25
13.1 Two-phase training explained	25
13.2 Train vs validation errors — why they differ	25
14. Model Evaluation	27
14.1 Headline: served Combined forecast vs random walk (untouched test set)	27
15. Baseline Comparison	29
16. Ensemble / Combined Prediction	30
16.1 Why an ensemble can reduce error — and its limits	30
16.2 The stacked-ensemble comparison	30
17. Final Prediction Flow	31
17.1 What happens when the user presses “Run prediction”	31
17.2 Failure behaviour (verified in code and tests)	31
18. UP / DOWN Logic	32
19. Streamlit Dashboard	33
19.1 What the user sees, in order	33
19.2 Messages, warnings and errors	33
20. Complete VS Code Project Structure	34
20.1 Top-level and configuration files	34
20.2 src/data — data layer	35
20.3 src/models — model layer	37
20.4 src/training and src/evaluation	37
20.5 src/experiments, src/inference, src/api, src/utils	39
20.6 app, scripts, tests, notebooks, results	40
21. File-to-File Connection Map	42
21.1 Serving path (what runs when you use the dashboard)	42
21.2 Training / evaluation path	42
21.3 Who imports whom (verified from the import statements)	42
22. Important Functions	43
23. Libraries and Technologies	44

24. Model Files / Saved Artefacts	45
25. Configuration and Environment	46
26. How to Run the Project	47
Step 1 — Install requirements	47
Step 2 — Configure environment	47
Step 3 — Prepare / download the dataset	47
Step 4 — Train models (if required)	47
Step 5 — Start Streamlit	47
Step 6 — Open the application	47
27. Complete End-to-End Example (real output)	48
Scenario A — the user selects Bitcoin and presses Run prediction	48
Scenario B — “Predict a Day” on an unseen test day (real output)	48
Scenario C — what the same Combined forecast looks like for the other assets (stored data)	48
28. Strengths of the Project	49
29. Limitations / Weaknesses	50
30. What Our System Does NOT Do	51
31. Common Viva Questions (50+ questions with answers)	52
31.1 Basic questions	52
31.2 Dataset questions	52
31.3 Machine learning questions	53
31.4 Feature engineering questions	54
31.5 Model questions	54
31.6 Time-series questions	55
31.7 Evaluation questions	56
31.8 Architecture / code questions	57
31.9 Limitations questions	57
31.10 Future work questions	58
32. “Explain My Project in 2 Minutes”	59
33. “Explain My Project in 5 Minutes”	60
34. Presentation Slide Content	61
35. System Architecture Diagram Explanation	63
36. Important Terminology	64
37. Code Explanation for Presentation	65
37.1 Close-time-safe macro alignment	65
37.2 Train-only scaling and chronological split	65
37.3 Target creation and date-based masks	65
37.4 Walk-forward evaluation	65
37.5 Two-phase LightGBM fit	66
37.6 The combined prediction	66
37.7 Single test-set evaluation	66
37.8 Conditional uncertainty band	66
37.9 Diebold–Mariano test	67
37.10 Run-prediction logic in the dashboard	67

37.11 Lagging the post-settlement features (futures)	67
38. Actual Results	68
38.1 Bitcoin	68
38.2 Gold	68
38.3 Silver	69
38.4 Uncertainty band ($\pm 1\sigma$, nominal coverage 68.3 %)	69
38.5 Experiment highlights (validation only, from results/experiments/)	70
39. Honest Project Assessment	72
40. Final “Student Cheat Sheet”	73
Appendix A — Verification Record	74

1. Executive Summary

1.1 In very simple English (what you would say to an examiner)

Our project looks at the daily prices of three assets — **Bitcoin, Gold and Silver** — and asks: “Using everything we know up to today, can a computer predict what tomorrow’s price will be?” We download years of daily prices from Yahoo Finance, turn them into numbers that describe the recent behaviour of the market (how much it moved lately, how volatile it is, whether it looks overbought, what stocks and the dollar did), and train six machine-learning models on that history. At the end we average the six models into one “combined” forecast. A Streamlit website shows the combined prediction for tomorrow as a price, says UP or DOWN, and draws an uncertainty band around it.

The most important thing we did is to test honestly. We compare every model against the simplest possible forecast — “tomorrow’s price will equal today’s price” (the **random walk**) — on days the models never saw. **Result: no model beats it.** That is not a failure of the code; it is the well-known fact that next-day price changes are almost pure noise. What our system adds is a correct, leak-free pipeline that can *prove* that, plus an uncertainty band that is reasonably well calibrated.

1.2 The questions an examiner will ask first

Question	Answer (verified in code)
What is the project?	A machine-learning system that forecasts the next trading day’s close of Bitcoin (BTC-USD), Gold futures (GC=F) and Silver futures (SI=F), with a Streamlit dashboard and a FastAPI endpoint.
What problem does it solve?	It gives a reproducible, honest answer to “can ML predict next-day prices, and does it beat a naive forecast?” and presents forecasts together with their measured reliability. It is an academic study, not a trading product.
What exactly is predicted?	The next-day log return $y = \ln(P_{t+1}/P_t)$, converted to a price with $P_{t+1} = P_t \cdot \exp(\hat{y})$.
Who could use it (theoretically)?	Students/researchers studying market predictability; an analyst wanting a volatility-aware range for tomorrow’s price. Not suitable for trading decisions (no skill vs. random walk).
Input?	Daily OHLCV of the asset + daily macro series (S&P 500, VIX, and for metals DXY, WTI crude, 10-y yield; Fear & Greed index for Bitcoin; gold return for Silver), all from Yahoo Finance / alternative.me.
What happens to the input?	Clean → compute 28–29 stationary features per day → scale (MinMax, fitted on training data only) → take the last 30 days as a window → six models each output a predicted log return.
Final output?	Predicted next-day close, UP/DOWN, % change, a $\pm 1\sigma$ price band, each model’s own prediction, and the held-out test metrics beside it.
Role of Streamlit?	It is only the front end: it triggers data refresh, calls <code>src/inference/prediction.py</code> , and reads result CSVs to display performance. No modelling happens in the UI file.

1.3 Technical summary

- **Data:** daily bars from 2018-01-01 to 2026-09-12 (Bitcoin, 3,177 raw rows) / 2026-09-11 (Gold, Silver, 2,186 raw rows each), Yahoo Finance via `yfinance`.
- **Features:** 29 (Bitcoin), 28 (Gold), 29 (Silver) stationary, backward-looking features; raw price levels are never model inputs.
- **Split:** one frozen chronological split by date: train $\leq$ 2025-09-10, validation $\leq$ 2026-02-17, test = 2026-02-18 → 2026-09-12 (207 Bitcoin days) / 2026-09-11 (143 Gold and Silver days).
- **Models:** Ridge, Random Forest, LightGBM, CatBoost, GRU, LSTM (six trained) + baselines Naive (random walk), Naive-Mean, ARIMA + a stacked-ensemble experiment.
- **Validation:** 4-fold expanding-window walk-forward inside train+val for tuning and model comparison; the test set is read by exactly one script (`src/evaluation/backtesting.py`).
- **Served forecast:** “Combined” = equal-weight mean of the six models’ predicted log returns. UP if the mean is > 0 .
- **Finding:** Combined RMSE (return space) is within ± 0.4 % of the random walk on the test set; Diebold–Mariano $p = 0.49 / 0.97 / 0.46$; directional accuracy 46.9 / 50.3 / 53.8 % (none significant).

2. Project Objectives

2.1 Primary objectives (supported by the implementation)

1. **Build an end-to-end ML pipeline** for three assets: data download → cleaning → feature engineering → training → evaluation → serving (all implemented; `make pipeline`).
2. **Predict the next trading day** for Bitcoin, Gold and Silver (target `log_return`, horizon 1 day — `config.py`).
3. **Evaluate honestly against a baseline:** every model is compared with the random walk and scored with return-space metrics, directional accuracy with a binomial test, and a Diebold–Mariano test (`src/utls/metrics.py`).
4. **Prevent data leakage:** causal features, train-only scaler, chronological split, “close-time rule” for macro data (`config.py`, `preprocessing.py`, 43 tests).
5. **Deliver a working dashboard** where the user presses one button and receives a combined forecast with its reliability context.

2.2 Secondary objectives

- Compare statistical (ARIMA), linear (Ridge), tree-based (Random Forest, LightGBM, CatBoost) and recurrent (GRU, LSTM) model families under identical conditions.
- Run design experiments on the validation data only: training-data size (E1), feature-group ablation (E2), 1-day vs 5-day horizon (E3), volatility target (E4) — `src/experiments/`.
- Provide a conditional $\pm 1\sigma$ uncertainty band from EWMA volatility and test its calibration (`results/band_calibration.csv`).
- Offer a “Predict a Day” demonstration on the unseen test period (`predict_for_date`) and a FastAPI service (`src/api/app.py`).
- Keep the work reproducible (seeds, frozen data, lock file, CI workflow).

2.3 Why each design choice exists

Choice	Reason (as implemented/documented in the code)
Machine learning	To test whether non-linear models can extract any pattern from engineered market features that a naive forecast cannot.
Historical data	ML learns from past examples (feature row → next-day return). Without history there is nothing to learn from; 2018–2026 gives ~2,000–3,000 daily examples per asset.
Feature engineering	Raw prices are non-stationary (their test values fall outside the training range). Ratios, returns and volatilities are scale-free, so they stay comparable over time.
Multiple models	Different families make different mistakes; comparing them is the scientific question, and the combination reduces model-specific variance.
Combining predictions	Equal-weight averaging needs no fitted weights, so nothing is selected on the test set (<code>prediction.py</code> docstring).
Dashboard	Makes the model usable and, more importantly, shows reliability metrics next to every forecast.

3. Problem Statement

Real-world problem. People want to know whether tomorrow's price of Bitcoin, Gold or Silver will go up or down. Forecasting daily prices is a classic hard problem in finance, and many student projects report impressive-looking accuracy that disappears once leakage is removed. This project asks the question properly.

3.1 Why prices are hard to predict (evidence from our own data)

- **Prices are non-stationary.** The ADF test on the price level cannot reject a unit root ($p = 0.97$ Bitcoin, 1.00 Gold, 0.97 Silver); on log returns it rejects strongly ($p \approx 0$). That is why we predict returns. (results/eda_summary.csv)
- **Returns carry almost no linear memory.** Lag-1 autocorrelation of daily returns on the training window: -0.053 (Bitcoin), -0.027 (Gold), -0.021 (Silver). ARIMA and lag features therefore have little to work with.
- **Fat tails.** Excess kurtosis of daily returns: 15.2 (Bitcoin), 3.6 (Gold), 5.2 (Silver); a normal distribution would be 0. Rare large moves dominate the squared error.
- **Noise dominates.** Daily return standard deviation (training window): 3.42 % Bitcoin, 0.97 % Gold, 1.91 % Silver, versus mean daily returns of 0.05–0.09 %.
- **Historical prices alone are not enough**, so we add volatility, momentum, macro and sentiment features — but our experiments (E2) show these add essentially nothing at a 1-day horizon.

3.2 Limitations of the approach

- The system only sees public daily data; news, geopolitics, order flow and central-bank decisions are not inputs.
- Relationships can change over time (regime shifts); the test window is only 143–207 days.
- Yahoo's futures series (GC=F, SI=F) is a continuous front-month series; contract rolls are not adjusted in our code.

Academic framing. This is a prediction *study* with a working prototype. It does not claim profit, perfect prediction or trading value. The dashboard footer and the API both return the disclaimer: "Research prototype for an academic project ... NOT financial advice."

4. Complete System Workflow

The flow below is the **actual** pipeline (verified against Makefile, scripts/retrain.py and the code). Differences from a textbook pipeline are marked ★.

```
Yahoo Finance (yfinance) + alternative.me Fear&Greed API
|
| src/data/data_collection.py::fetch_asset      src/data/external_data.py::fetch_all_external_data
| (complete-bar rule: market_calendar.drop_incomplete_bars)
data/raw/*.csv (bitcoin, gold, silver, dxy, crude_oil, vix, tn timer, sp500, fear_greed)
|
| src/data/preprocessing.py::DataCleaner.load_data / clean_data
| de-duplicate → calendar handling → log_return → indicators → relative features
data/processed/<asset>_features.csv (levels kept for charts + model features)
|
| DataCleaner.normalize_and_split ★ split by FIXED DATES; MinMaxScaler fitted on TRAIN only
|
data/processed/<asset>_{train,val,test}_scaled.csv + data/models/<asset>_scaler.pkl
|
| preprocessing.build_dataset(asset) ★ the ONE loader; target created here (next-day log return, z-scored with train stats)
|
windows X (n, 30, F) / last-row Xt (n, F) / y – split by the dates the TARGET covers
|
|— tune_models.py → 4-fold walk-forward grid search (train+val only) → results/tuning/best_params.json
|— train_models.py → phase A (train, monitor val) + phase B (refit train+val) → data/models/*
|— ensemble_model.py → stacked meta-model on out-of-fold predictions (EXPERIMENT, not served)
|
src/evaluation/backtesting.py ★ the ONLY reader of the test split
|
| → results/cv_results.csv, final_test_results.csv, band_calibration.csv, predictions/*.csv, FINAL_RESULTS.md,
| data/models/model_status.json
|
src/inference/prediction.py → Combined = equal-weight mean of 6 models' predicted log returns
|
| →  $P^* = P_t \cdot \exp(r^*)$ ; UP if  $r^* > 0$ ;  $\pm 1\sigma$  band  $P_t \cdot \exp(r^* \pm \sigma_t)$ 
|
app/streamlit_app.py (dashboard)      src/api/app.py (FastAPI)
```

4.1 Stage-by-stage table

Stage	File → function	Data in	Data out	Why necessary
Download prices	data_collection.py → fetch_asset	ticker, start date	data/raw/<asset>_data.csv (timestamp, open, high, low, price, volume)	Source data
Download macro/sentiment	external_data.py → fetch_yfinance_external, fetch_fear_greed_index	tickers / API URL	dxy, crude_oil, vix, tn timer, sp500, fear_greed CSVs	Extra information for features
Complete-bar rule	market_calendar.py → drop_incomplete_bars	raw rows	rows with finished bars only	Yahoo returns the running day as if finished
Cleaning	preprocessing.py → DataCleaner.clean_data	raw CSV	sorted, de-duplicated, positive prices, crypto calendar filled	Valid time series
Feature engineering	DataCleaner.add_*, _align_external	cleaned prices + external CSVs	<asset>_features.csv	Informative, stationary inputs
Split + scaling	DataCleaner.normalize_and_split, save_data	features	scaled train/val/test CSVs + scaler.pkl	Leak-free evaluation
Target + windows	build_dataset, _task_target, _make_windows	scaled CSVs + features	dict with X, Xt, y (standardised), dates, prices	Supervised-learning format
Tuning	tune_models.py → tune	train+val	best_params.json, per-config CSVs	Choose hyper-parameters without the test set
Training	train_models.py → train_asset	train, val	saved models, train_val_metrics.csv, feature_importance.csv	Fit the deployed models
Evaluation	backtesting.py → main	saved models + test split	result tables, prediction CSVs, model_status.json	One honest test-set evaluation
Prediction	prediction.py → predict_next_day	latest features + models	dict: price, return, direction, band, members	Serve a forecast
Display	streamlit_app.py	prediction dict + result CSVs	web page	User interface

★ **Differences from a generic ML pipeline.** (1) The target is created inside build_dataset, not in the cleaner. (2) The split is by fixed calendar dates stored in config.py, not by percentages. (3) There is no “model selection” in the served forecast: all six models are averaged. (4) The stacked ensemble exists but is an experiment and is **not** served.

5. Dataset — Complete Details

5.1 Source

Item	Detail (from code and files)
Price source	Yahoo Finance through the Python library yfinance (pinned 1.2.0): <code>yf.Ticker(ticker).history(start=...)</code> in <code>data_collection.py::fetch_asset</code> .
Macro source	Yahoo Finance: DX-Y.NYB (US Dollar Index), CL=F (WTI crude), ^VIX, ^TNX (10-year yield), ^GSPC (S&P 500) — <code>external_data.py::EXTERNAL_SOURCES</code> .
Sentiment source	Crypto Fear & Greed index from the free alternative.me API (https://api.alternative.me/fng/?limit=0&format=json), no API key, fetched with <code>urllib</code> .
Automatic or local?	Both. <code>make_collect-data</code> downloads into <code>data/raw/</code> ; the repo also stores the downloaded CSVs locally (git-ignored except a 3-year backup in <code>data/raw_3y_backup/</code>). All reported results use the frozen files in <code>data/raw/</code> .
Live refresh	<code>sync_live_data.py</code> downloads into <code>data/raw_live/</code> and writes <code><prefix>_live_features.csv</code> ; it never overwrites <code>data/raw/</code> or the scaled splits.
Historical?	Yes. <code>DATA_START_DATE = 2018-01-01</code> (<code>config.py</code>).
Frequency	Daily bars (one row per trading day; Bitcoin every calendar day).

5.2 The three assets

Asset	Ticker	Type	Calendar	Raw rows	First date	Last date	Rows after features
Bitcoin	BTC-USD	crypto	every day (7 days/week)	3,177	2018-01-01	2026-09-12	3,147 (2018-01-31 → 2026-09-12)
Gold	GC=F	commodity (COMEX futures)	exchange days only	2,186	2018-01-02	2026-09-11	2,156 (2018-02-15 → 2026-09-11)
Silver	SI=F	commodity (COMEX futures)	exchange days only	2,186	2018-01-02	2026-09-11	2,156 (2018-02-15 → 2026-09-11)

The ~30 rows lost between “raw” and “features” are the warm-up rows removed by `dropna()` after rolling windows (up to 30 days) and indicator calculations. Gold and Silver are Yahoo’s **continuous front-month futures** series, so they are not spot prices.

5.3 External series row counts (data/raw/)

File	Rows	Date range
<code>dxy_data.csv</code>	2,187	2018-01-02 → 2026-09-11
<code>crude_oil_data.csv</code>	2,187	2018-01-02 → 2026-09-11
<code>vix_data.csv</code>	2,187	2018-01-02 → 2026-09-11
<code>tnx_data.csv</code>	2,185	2018-01-02 → 2026-09-11
<code>sp500_data.csv</code>	2,185	2018-01-02 → 2026-09-11
<code>fear_greed_data.csv</code>	3,143	2018-02-01 → 2026-09-13

5.4 Raw columns (every column of the OHLCV files)

Column	Meaning	Type	Role
<code>timestamp</code>	Date of the daily bar (timezone removed)	<code>datetime</code>	index/metadata
<code>open</code>	First price of the day	<code>float</code>	kept in <code>*_features.csv</code> for charts; not a model input (LEVEL_COLUMNS)
<code>high</code>	Highest price of the day	<code>float</code>	used to compute ATR, ADX, <code>hl_range</code> ; level itself not a model input
<code>low</code>	Lowest price of the day	<code>float</code>	same as high
<code>price</code>	Closing price (Yahoo “Close”) — the series we forecast	<code>float</code>	used to compute returns and indicators; level not a model input; target is derived from it
<code>volume</code>	Traded volume	<code>float</code>	Bitcoin only: <code>log_volume_change</code> feature. Futures volume is unused (front-month contract volume jumps at rolls)

External files have columns timestamp, price (and timestamp, fear_greed for the sentiment index). The processed feature files contain 42 (Bitcoin) / 41 (Gold) / 42 (Silver) columns, of which 29 / 28 / 29 are model features; the rest are level columns (open, high, low, price, volume, EMA_14, EMA_30, BB_Mid, BB_Upper, BB_Lower, ATR, MACD, MACD_Signal) kept for charts.

5.5 Data-quality handling (what the code really does)

Topic	What exists in the code	Result on our data
Missing values	Raw: <code>dropna(subset=["price"])</code> . After features: rows with NaN are dropped (<code>dropna</code>), then <code>validate_features</code> raises an error if any NaN/inf remains or fewer than 100 rows. Macro return gaps → filled with 0; missing Fear & Greed → 50. Crypto calendar gaps would be forward-filled.	0 nulls in all raw OHLCV files; Bitcoin has no calendar gaps (3,177 rows = 3,177 days).
Duplicates	<code>drop_duplicates(subset=["timestamp"])</code> then sort (<code>clean_data</code>); external series: <code>s[~s.index.duplicated()]</code> .	0 duplicate timestamps found in raw files (checked).
Invalid prices	<code>df[df["price"] > 0]</code> .	None removed.
Outliers	No outlier removal or winsorising of prices/returns (fat tails are real information). Only <code>log_volume_change</code> is clipped to <code>[-3, 3]</code> .	Test-set values can fall outside the training range after scaling: 0.02 % (Bitcoin), 3.4 % (Gold), 2.6 % (Silver) of test cells lie outside <code>[0, 1]</code> . No clipping is applied.
Incomplete current bar	<code>drop_incomplete_bars</code> removes the still-running day (UTC day for crypto; before 17:15 ET for futures/US series).	Frozen files end on complete bars.

5.6 Scaling and normalisation

- **Scaler:** `sklearn.preprocessing.MinMaxScaler(feature_range=(0, 1))` — `DataCleaner.normalize_and_split`.
- **Fitted on:** training rows only ($\leq 2025-09-10$); applied unchanged to validation and test. Saved as `data/models/<prefix>_scaler.pkl` (joblib) and reloaded at prediction time (`prediction._window`).
- **Applied to:** the model-feature columns only (not price levels).
- **Why:** neural networks (GRU/LSTM) and Ridge need inputs on similar scales; tree models do not care but receive the same inputs for a fair comparison.
- **Target scaling (separate):** the target is z-scored with the training-window mean and standard deviation (`target_mean`, `target_std` in `build_dataset`; stored in `model_status.json`) and converted back with $\text{inv}(v) = v \cdot \text{sd} + \text{mu}$.
- **Verified:** training data scales to exactly `[0, 1]`; the test set contains values outside `[0, 1]` (expected — proof that test statistics were not used).

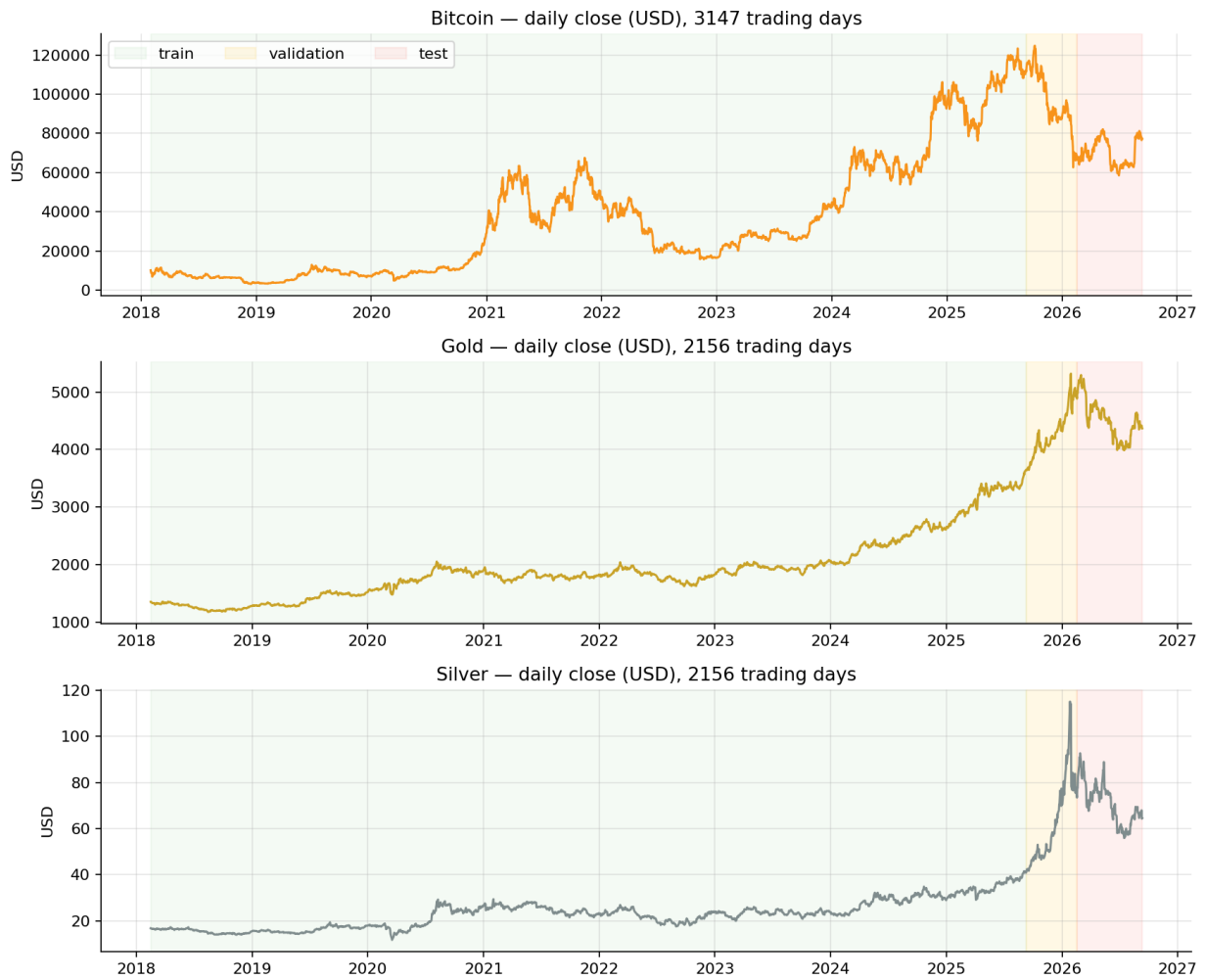


Figure 5.1 — Price history of the three assets with the train / validation / test shading (results/figures/price_history.png).

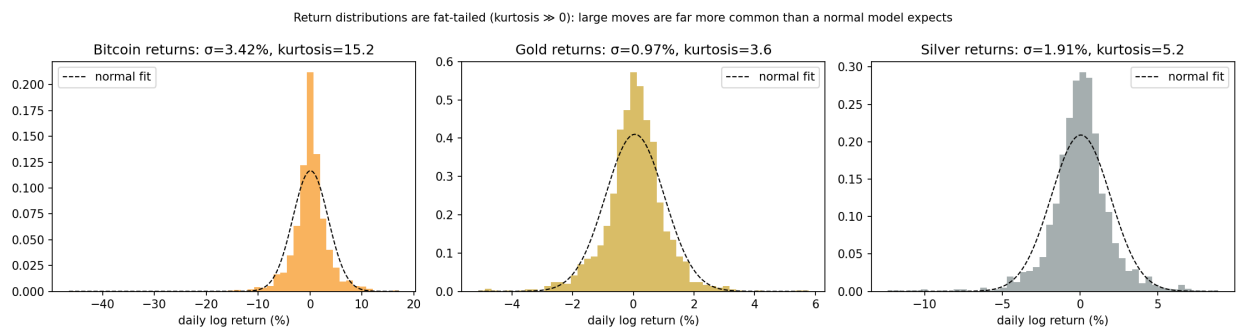


Figure 5.2 — Daily return distributions are fat-tailed (kurtosis $\gg 0$). Source: src/data/eda.py.

6. Target Variable

6.1 What is predicted

The target is the **next-day log return**, not the price:

```
y_t = ln( P_{t+1} / P_t )          # P = daily close; config.TARGET_COL = "log_return", PREDICTION_HORIZON_DAYS = 1
```

In words: *today's information (row t) → predict how much tomorrow's close differs from today's, on a logarithmic scale*. The price forecast is then $P_{t+1} = P_t \cdot \exp(\hat{y}_t)$ (src/utils/inverse_transform.py, prediction.py). A log return of +0.0016 means about +0.16 %.

6.2 Where and how it is created

- **Function:** src/data/preprocessing.py::_task_target(log_return, task) for the served task return_1d → fwd = r.shift(-1), i.e. the log return of the *next* row, aligned to the date of day t.
- **Called from:** build_dataset ("the ONE loader used by tuning, training, stacking, evaluation and inference").
- **Column name:** the feature column log_return is today's return (an input); the target is its one-step-ahead shifted copy and is held in the dataset dict as y_real_<split> (real units) and y_<split> (standardised).
- **Horizon:** 1 trading day (Bitcoin: next calendar day; Gold/Silver: next exchange day; weekends/holidays are skipped for futures).
- **Rows lost:** the very last row of each asset has no "next day", so its target is NaN and it is dropped (valid = target.notna()); the first 29 rows of each split chain are lost as window warm-up (SEQ_LEN = 30).
- **Other tasks (experiments only):** return_5d, vol_5d, vol_22d exist in config.TASKS and are used only in src/experiments/. The served model is return_1d.

6.3 Why this target

- Prices are non-stationary (ADX/ADF evidence in Ch. 3), returns are stationary — a model trained on returns generalises across price regimes.
- It makes the baseline explicit: the random walk is simply $\hat{y} = 0$.
- The price forecast is obtained exactly and deterministically from the return (tested by test_true_target_reconstructs_actual_close_exactly).
- Predicting the raw price would make any model look great (R^2 of price ≈ 0.95 for the naive forecast) while telling us nothing — see the note in src/utils/metrics.py.

Viva trap: "Your MAPE is only 1.5 % — is that good?" Answer: the naive "tomorrow = today" forecast scores the same 1.5 % (Bitcoin test MAE \$1,066 naive vs \$1,064 Combined). MAPE on prices rewards any forecast that stays near today's price.

7. Feature Engineering — every feature of the final models

All features are computed in `src/data/preprocessing.py::DataCleaner` on the **full** history of each asset *before* the split. This is safe because every feature at row `t` is computed only from information available at the moment the day-`t` close is fixed (rolling windows and EWMA are backward-looking; shifts are positive). Chapter 8 proves this.

7.1 Which features each asset uses

Asset	Count	Common 23 features	Asset-specific features
Bitcoin	29	all 23 below	dow_sin, dow_cos, log_volume_change, sp500_return, vix_return, fear_greed
Gold	28	all 23 below	dxy_return, oil_return, tnx_return, sp500_return, vix_return
Silver	29	all 23 below	dxy_return, oil_return, tnx_return, sp500_return, vix_return, gold_return

Common 23: `log_return`, `RSI`, `ADX`, `ROC`, `ema14_ratio`, `ema30_ratio`, `macd_norm`, `macd_hist_norm`, `bb_pctb`, `bb_width`, `atr_norm`, `hl_range`, `return_1d`, `return_2d`, `return_5d`, `return_10d`, `return_20d`, `volatility_10d`, `volatility_30d`, `rv_1d`, `rv_5d`, `rv_22d`, `ewma_vol`. The order above is exactly the order of the columns fed to the models (`status["features"]` in `model_status.json`).

7.2 Feature dictionary

Feature	Meaning	Calculation (code)	Why it may help	Type
<code>log_return</code>	Today's log return	<code>ln(P_t / P_{t-1})</code>	Latest move; momentum/reversal	price-based
<code>return_1d</code>	Yesterday's return	<code>log_return.shift(1)</code>	Short-term autocorrelation	lagged
<code>return_2d</code>	Return 2 days ago	<code>log_return.shift(2)</code>	same	lagged
<code>return_5d</code>	Return 5 days ago	<code>log_return.shift(5)</code>	weekly pattern	lagged
<code>return_10d</code>	Return 10 days ago	<code>log_return.shift(10)</code>	two-week pattern	lagged
<code>return_20d</code>	20-day cumulative return	<code>log_return.rolling(20).sum()</code>	Monthly momentum / reversal	rolling
<code>RSI</code>	Relative Strength Index (0-100)	Wilder smoothing, window 14: $100 - 100/(1 + \text{avg gain/avg loss})$	Overbought/oversold	technical
<code>ADX</code>	Average Directional Index	<code>ta.trend.ADXIndicator</code> , window 14	Trend strength (not direction)	technical
<code>ROC</code>	Rate of change, 12 days	<code>ta.momentum.roc</code> , window 12 (percent)	Medium-term momentum	technical
<code>ema14_ratio</code>	Price relative to its 14-day EMA	$P / \text{EMA14} - 1$ (EMA span 14, <code>adjust=False</code>)	Distance from trend	technical, scale-free
<code>ema30_ratio</code>	Price relative to 30-day EMA	$P / \text{EMA30} - 1$	same	technical, scale-free
<code>macd_norm</code>	MACD divided by price	$(\text{EMA12} - \text{EMA26}) / P$	Trend momentum, scale-free	technical
<code>macd_hist_norm</code>	MACD histogram / price	$(\text{MACD} - \text{Signal}) / P$; Signal = EMA9 of MACD	Momentum acceleration	technical
<code>bb_pctb</code>	Bollinger %B	$(P - \text{lower}) / (\text{upper} - \text{lower})$; 20-day, 2σ	Position inside the bands	technical
<code>bb_width</code>	Bollinger band width	$(\text{upper} - \text{lower}) / \text{middle}$	Volatility state	technical / volatility
<code>atr_norm</code>	Average True Range / price	<code>ta.ATR(14) / P</code> (futures: lagged 1 session)	Volatility incl. gaps	volatility
<code>hl_range</code>	Daily high-low range / price	$(\text{high} - \text{low}) / P$ (futures: lagged 1 session)	Intraday volatility	volatility
<code>volatility_10d</code>	10-day return std	<code>log_return.rolling(10).std()</code>	Short-term volatility	rolling
<code>volatility_30d</code>	30-day return std	<code>log_return.rolling(30).std()</code>	Medium-term volatility	rolling
<code>rv_1d</code>	Realised vol (daily)	$ \text{log_return} = \sqrt{r^2}$	HAR volatility component	volatility
<code>rv_5d</code>	Realised vol (weekly)	$\sqrt{\text{mean of last 5 } r^2}$	HAR weekly component	volatility
<code>rv_22d</code>	Realised vol (monthly)	$\sqrt{\text{mean of last 22 } r^2}$	HAR monthly component	volatility
<code>ewma_vol</code>	RiskMetrics EWMA volatility	$\sqrt{\text{EWMA of } r^2, \lambda = 0.94, \text{ min 22 obs }}$	Best simple vol forecast; also σ_t of the band	volatility

Feature	Meaning	Calculation (code)	Why it may help	Type
dow_sin, dow_cos	Cyclical day of week (Bitcoin only)	$\sin/\cos(2\pi \cdot \text{dow}/7)$	Weekday effects (7-day market)	calendar
log_volume_change	Log change in volume (Bitcoin only)	$\ln(V_t / V_{t-1})$, clipped to $[-3, 3]$, 0 filled	Activity/attention	volume-based
sp500_return	S&P 500 log return	$\ln(S_t/S_{t-1})$ on its own calendar, aligned (see 7.3)	Risk-on/risk-off	macro
vix_return	VIX log return	same	Fear gauge change	macro
fear_greed	Crypto Fear & Greed value (0–100), Bitcoin only	raw index value, aligned same-day; missing → 50	Crypto sentiment	sentiment
dxy_return	US-dollar index return (metals)	previous-day value (see 7.3)	Gold/Silver vs. dollar	macro
oil_return	WTI crude return (metals)	previous-day value	Commodity co-movement	macro
tnx_return	10-year yield return (metals)	previous-day value	Real-rate channel for gold	macro
gold_return	Gold log return (Silver only)	same-day gold return (both settle ~13:30 ET)	Gold-silver co-movement	macro / cross-asset

7.3 The close-time rule (the most important design idea in the project)

A feature at row t may only use information known **when the asset's day- t close is fixed**. Because assets close at different times, macro data is aligned differently (`config.EXTERNAL_SAME_DAY`, `DataCleaner._align_external`):

Asset	Daily close fixed at	Macro alignment	High/Low features
Bitcoin (BTC-USD)	00:00 UTC (after every US market close)	Same-day S&P 500, VIX, Fear & Greed values (forward-filled onto the Bitcoin calendar)	Same-day (no lag)
Gold / Silver futures	13:25–13:30 ET COMEX settlement (before the S&P 16:00, VIX 16:15, DXY 17:00 closes)	Previous trading day's value: last external date strictly before t (<code>searchsorted(..., side="left") - 1</code>)	hl_range, atr_norm, ADX shifted one session (Yahoo's High/Low span the whole Globex session, beyond the settlement)

History worth knowing for the viva: an intermediate version used same-day macro data for the metals and showed Silver at 62 % directional accuracy. The final audit traced this to the close-time leak, fixed it, re-ran everything, and the 62 % disappeared (archived in `results/archive_sameday_macro_leak/`). Being able to say this shows the audit works.

7.4 Quirks you should know (verified in code)

- For Bitcoin, the S&P 500 and VIX *return series* is forward-filled onto weekends, so Saturday and Sunday repeat Friday's return rather than 0. This is a data-representation quirk, not leakage (only past values are pulled forward).
- Silver's `gold_return` is a same-day feature; the code comment argues the 5-minute settlement difference between gold (13:30 ET) and silver (13:25 ET) is negligible. It is a small residual assumption.
- `log_return` (today's return) is an input; the target is tomorrow's. They are different columns.

7.5 Features that are NOT model inputs

- `LEVEL_COLUMNS` (`config.py`): open, high, low, price, volume, `EMA_14`, `EMA_30`, `BB_Mid`, `BB_Upper`, `BB_Lower`, `ATR`, `MACD`, `MACD_Signal` — kept in `*_features.csv` for charts only.
- Futures volume (unusable: front-month contract volume), Fear & Greed for the metals, calendar features for the metals, DXY/oil/10-y yield for Bitcoin.
- No news, no on-chain data, no fundamental data.

What the models actually rely on (feature importance from `results/feature_importance.csv`; each tree model ranks differently, which is itself a sign that no stable signal exists): Bitcoin — `atr_norm`, `sp500_return`, `return_1d`, `log_return`; Gold — `return_20d`, `tnx_return`, `dxy_return`, `oil_return`; Silver — `ema14_ratio`, `macd_hist_norm`, `volatility_30d`, `gold_return`. The feature-ablation experiment (E2) shows that dropping any

feature group changes the validation RMSE by only about -0.4% ... $+0.05\%$.

8. Data Leakage Audit

Data leakage means information from the future (or from the test set) enters the model's inputs, making results look better than they can ever be in reality. This chapter lists every place it could happen, what the code does, and the evidence. Verdict: **no look-ahead leakage found in the final pipeline**; one real leak occurred earlier and was fixed.

Question	Finding	Evidence
Can future information enter the features?	No. All rolling windows are backward (rolling(n), ewm, positive shift); no centered windows, no negative shifts in feature code (the only negative shift is the target).	test_no_lookahead_in_any_feature (changes the last bar, checks all earlier features unchanged, for Bitcoin and Gold). My own run: rebuilt features from raw data truncated at 2026-06-01 → max absolute difference vs full-data features 1.5e-11 (Bitcoin), 9e-13 (Gold), 3e-13 (Silver) = identical.
Are rolling features correct?	Yes — lags are strictly past, RSI bounded 0–100, Bollinger bands ordered.	test_lagged_returns_are_strictly_past, test_rsi_bounds_and_bollinger_ordering
Is the target handled correctly?	The target is the next row's log return and never appears in the input window (window i covers rows i...i+29; target is row i+30).	test_create_sequences_target_is_next_row_and_never_in_window, test_true_target_reconstructs_actual_close_exactly
Is scaling fitted only on training data?	Yes. MinMaxScaler.fit(train_df); val/test only transform. Target z-score uses training targets only.	test_scaler_fitted_on_train_only; I verified train scales to [0,1] and the test set exceeds that range (0.02 % / 3.4 % / 2.6 % of cells).
Is validation/test kept unseen?	Tuning and CV use only train+val (trainval_arrays); only backtesting.py reads X_test. Splits are defined by the dates the target covers.	test_split_is_chronological_and_disjoint; code review of tune/train/cross_validation (no test access).
Feature engineering before or after the split?	Before — on the full history. This is leak-free only because every feature is causal; scaling (which would leak) is done after the split.	Truncation test above.
Does any preprocessing use the whole dataset?	EDA statistics use the training window only. The only whole-history operations are causal feature calculations.	eda.py: train = df.loc[:TRAIN_END]
Macro data timing (the close-time leak)?	Fixed. Metals use previous-day macro values and lagged High/Low features; Bitcoin may use same-day values.	test_commodity_external_features_are_previous_day_values, test_commodity_high_low_features_are_lagged_one_session; archive of the leaky run.
Hyper-parameter / early-stopping leakage?	Tuning on walk-forward folds inside train+val. Early stopping uses a monitor window (phase A: validation; CV folds: last 15 % of the fold's training window). The fold's validation block is only predicted, never fitted.	cross_validation.cv_evaluate, registry two-phase fitting.
Incomplete bar of the current day?	Dropped at download (complete-bar rule).	test_drop_incomplete_bars_removes_todays_running_row
Multi-day targets (experiments)	Purged/embargoed: samples split by the dates their targets cover.	test_task_targets_are_future_only_and_embargoed

8.1 Residual risks (honest list)

- **Test-set reuse across project iterations.** The final pipeline reads the test split once, but earlier versions of the project (3-year data; same-day-macro version) were evaluated on the same calendar window, and the leak fix was motivated by an anomalous test result. The final selection rules use validation only, but the p-values on the test set are therefore not “pristine” in the strict sense. Say so if asked.
- **Silver's gold_return is same-day** (5-minute assumption).
- **Vendor revisions:** Yahoo revises some historical closes between downloads (the repo measured up to 0.75 % on one Bitcoin day); results are tied to the frozen files in data/raw/.
- **Continuous futures:** front-month roll jumps are not adjusted, producing artificial returns on roll days.

Conclusion. No future information reaches the features of the final pipeline; the earlier leak was found by the project's own audit and fixed. The evidence is both unit tests (43 passing) and an independent truncation test run for this guide.

9. Train / Validation / Test Split

9.1 The frozen split

Defined once in config.py and used by every script:

```
TRAIN_END = "2025-09-10"      # train: t ≤ TRAIN_END
VAL_END   = "2026-02-17"      # validation: TRAIN_END < t ≤ VAL_END
# test: everything after VAL_END (touched once, by src/evaluation/backtesting.py)
```

Asset	Split	Rows (scaled CSV)	Date range of rows	Samples (windows)	Share of samples
Bitcoin	Train	2,780	2018-01-31 → 2025-09-10	2,750	88.2 %
	Validation	160	2025-09-11 → 2026-02-17	160	5.1 %
	Test	207	2026-02-18 → 2026-09-12	207	6.6 %
Gold	Train	1,904	2018-02-15 → 2025-09-10	1,874	88.1 %
	Validation	109	2025-09-11 → 2026-02-17	109	5.1 %
	Test	143	2026-02-18 → 2026-09-11	143	6.7 %
Silver	Train	1,904	2018-02-15 → 2025-09-10	1,874	88.1 %
	Validation	109	2025-09-11 → 2026-02-17	109	5.1 %
	Test	143	2026-02-18 → 2026-09-11	143	6.7 %

“Samples” are the supervised-learning examples after target creation: each is a 30-day window ending on day t with the target of day $t+1$. Test samples have **target dates** 2026-02-18 → 2026-09-12 (Bitcoin) and 2026-02-18 → 2026-09-11 (metals); their input windows end one day earlier. The split is by the dates the *target* covers: a training target never reaches past TRAIN_END, a validation target lies in (TRAIN_END, VAL_END], a test target after VAL_END.

9.2 Strategy used: chronological + expanding-window walk-forward

Question	Answer
Random split?	No. No shuffling anywhere (Keras shuffle=False; splits by date).
Chronological split?	Yes — train → validation → test in time order.
TimeSeriesSplit?	Yes , for the walk-forward folds: walk_forward_splits wraps sklearn.model_selection.TimeSeriesSplit(n_splits=4) (cross_validation.py).
Walk-forward / expanding window?	Yes — 4 expanding folds inside train+val (CV_FOLDS = 4): fold k trains on everything before its block and predicts the block. For Bitcoin the blocks are 582 samples each; for the metals ≈ 396 .
Where is each split used?	Train: fit models. Validation: early-stopping/phase-A metrics and part of the walk-forward folds. Walk-forward folds (train+val): hyper-parameter tuning and model comparison. Test: one final evaluation.

```
Walk-forward folds (train+val only), expanding window:
|----- train -----|-- fold 1 --|
|----- train -----|-- fold 2 --|
|----- train -----|-- fold 3 --|
|----- train -----|-- fold 4 --|
then (not used in any of the above):          |----- TEST -----|
```

9.3 Why chronological splitting is essential

Time-series examples are correlated in time (volatility clusters). A random split would put days next to each other in train and test, letting the model “peek” at the surrounding market state, which inflates scores. Chronological order mimics real use: the model is always trained on the past and judged on the future.

9.4 Limitations of this validation design

- The validation window (109–160 days) and test window (143–207 days) are short; at $\sim 50\%$ direction rate the standard error of a hit-rate on 143 days is ≈ 4 percentage points, so differences of a few points are not detectable.
- All three assets share the same calendar split, so results are not independent across assets (the same 2026 market regime).

- The test period (Feb–Sep 2026) is one market regime; Bitcoin’s buy-and-hold return was +14.5 % and Gold’s –10.6 % in it.

10. Machine-Learning Models

The model registry (`src/models/registry.py`) gives every model the same interface (`fit`, `predict`, `save`, `load`) so all are trained and compared on **identical samples**. There are **six trained models** (served), **three baselines** and **one stacked-ensemble experiment**.

Group	Model	Class in registry.py	Input given to the model	Served?
Baseline	Naive (random walk)	NaiveModel	none — predicts $\hat{r} = 0$	reference only
Baseline	Naive-Mean	NaiveMean	none — predicts the training-mean return	reference only
Baseline	ARIMA(p,0,q)	ARIMAModel	the return series itself (order chosen by AIC)	reference only
Linear	Ridge	RidgeModel	today's feature row (n, F)	yes (member)
Tree ensemble	Random Forest	RandomForestModel	today's feature row (n, F)	yes (member)
Boosting	LightGBM	LightGBMModel	today's feature row (n, F)	yes (member)
Boosting	CatBoost	CatBoostModel	today's feature row (n, F)	yes (member)
Recurrent NN	GRU	GRUModel	30-day window (n, 30, F)	yes (member)
Recurrent NN	LSTM	LSTMModel	30-day window (n, 30, F)	yes (member)
Meta-model	Stacked (non-negative Ridge)	ensemble_model.StackedModel	out-of-fold predictions of Ridge, LightGBM, CatBoost, GRU	no — experiment

All models predict the same thing: the **standardised next-day log return** $\hat{y}$ (z-score with the training mean/std). The prediction is converted back with $\hat{r} = \hat{y} \cdot sd + \mu$. “F” is 29 / 28 / 29 features for Bitcoin / Gold / Silver. Tabular models see only the *last row* of each window (today's indicators); the recurrent models see all 30 rows.

10.1 Naive (random walk) — the baseline

In simple English. It simply says “tomorrow's price = today's price”, i.e. predicted return = 0. It needs no learning.

Technically. For return tasks `NaiveModel.predict` returns a constant 0 (`dataset["naive_<split>"]` holds zeros). Under the random-walk hypothesis the best forecast of tomorrow's price is today's price. Its metrics are computed from the actual returns: $RMSE = \sqrt{\text{mean}(r^2)}$.

Why we used it. It is the scientifically correct yardstick: a model is only useful if it beats it. Diebold-Mariano tests in `metrics.py` compare every model with this forecast.

Strengths	Limitations	Role in our project
Impossible to “leak”; zero variance; very hard to beat at a 1-day horizon.	Says nothing about direction.	The benchmark every other number is compared with.

10.2 Naive-Mean and ARIMA — the other baselines

In simple English. Naive-Mean always predicts the average daily return seen in training (a small positive drift). ARIMA is the classic statistics model that predicts the next value from recent past values and past errors.

Technically. Naive-Mean: $r^{\wedge} = \text{mean}(y_{\text{train}})$. ARIMA(p,0,q) on the standardised return series ($d = 0$ because returns are already differenced); order chosen by AIC over $p, q \in \{0..3\}$ on the training window; one-step-ahead forecasts are made walk-forward — after each day the true value is appended with `res.append([v], refit=False)` (no refit). Chosen orders on the final test run: Bitcoin (2,0,0), Gold (1,0,1), Silver (2,0,3).

Why we used it. To show whether ML adds anything beyond constant and classical statistical forecasts.

Strengths	Limitations	Role in our project
Simple, interpretable, standard in time-series courses.	Linear and univariate (ARIMA ignores all other features).	Baselines in every comparison table; not served.

10.3 Ridge regression

In simple English. A weighted sum of the features, like a straight-line formula, with a “penalty” that keeps the weights small so the model does not chase noise.

Technically. $\hat{y} = \beta_0 + \sum \beta_j x_j$, with β chosen to minimise $\sum (y - \hat{y})^2 + \alpha \cdot \sum \beta_j^2$. Implemented with `sklearn.linear_model.Ridge(alpha=...)` on today’s scaled feature row. `feature_importance = |coefficient|`.

Why we used it. Simplest learnable model; strong regularisation is exactly what a low-signal problem needs.

Strengths	Limitations	Role in our project
Fast, stable, hard to over-fit, interpretable.	Only linear relationships.	Member of the Combined forecast. Tuned $\alpha = 100$ for all three assets (the top of the search grid, i.e. the data prefers maximum shrinkage).

10.4 Random Forest

In simple English. Hundreds of small decision trees each vote on tomorrow’s return; the answer is the average of their votes. Each tree sees a slightly different random sample, so they disagree in useful ways.

Technically. `sklearn.ensemble.RandomForestRegressor(random_state=42, n_jobs=-1, n_estimators=300, max_depth=3, min_samples_leaf=30)`. Bagging (bootstrap rows) + random feature subsets at splits + averaging reduces variance. No early stopping; in phase B it is refit on train+val.

Why we used it. A non-linear, assumption-free model that handles interactions between features.

Strengths	Limitations	Role in our project
Robust to outliers and scaling; little tuning needed.	Cannot extrapolate beyond training range; shallow trees needed here or it over-fits noise.	Member of the Combined forecast; identical tuned parameters for all assets.

10.5 LightGBM

In simple English. Builds trees one after another; each new tree tries to fix the mistakes of all earlier trees. It is the “boosting” idea, implemented for speed.

Technically. `lightgbm.LGBMRegressor` (gradient-boosted decision trees, leaf-wise growth). Phase 1: fit on train with early stopping (30 rounds) on validation to find the number of trees; phase 2: refit on train+val with that tree count. Final tree counts: Bitcoin 81, Gold 10, Silver 10. `feature_importance = gain share`.

Why we used it. State-of-the-art tabular learner; tests whether boosting finds structure that simpler models miss.

Strengths	Limitations	Role in our project
Fast, handles non-linearity and mixed feature types.	Easy to over-fit small noisy data; stops after very few trees here because the validation error does not improve.	Member of the Combined forecast. Note: <code>subsample = 0.8</code> is set, but LightGBM applies row subsampling only when <code>subsample_freq > 0</code> , which the code does not set (per LightGBM docs, bagging is then off); <code>colsample_bytree = 0.8</code> is active.

10.6 CatBoost

In simple English. Another boosting method (trees that correct earlier trees) that uses “symmetric” trees, which makes it naturally resistant to over-fitting.

Technically. `catboost.CatBoostRegressor` with early stopping (50 rounds, `use_best_model=True`) on validation to pick the iteration count; refit on train+val with that count. Final iterations: Bitcoin 50, Gold 606, Silver 82. Saved as `.cbm` + a JSON side-car (params, fit_info).

Why we used it. A strong boosting alternative that is robust with little tuning; gives a second boosted view.

Strengths	Limitations	Role in our project
Good default behaviour, less over-fitting than other boosters.	Slower than LightGBM; still limited by noise.	Member of the Combined forecast; chosen as the best single model by validation for Bitcoin and Gold.

10.7 GRU and LSTM (recurrent neural networks)

In simple English. These networks read the last 30 days in order, one day at a time, and keep a “memory” of what they have seen. They then output one number: tomorrow’s predicted return.

Technically. Architecture (identical for both, `model_gru.py`, `model_lstm.py`): `Input(30, F) → GRU(32) | LSTM(32) → Dropout(p) → Dense(16, ReLU) → Dense(1)`, Adam optimiser, MSE loss. Parameter counts: GRU 6,593 (Bitcoin/Silver) / 6,497 (Gold); LSTM 8,481 / 8,353. Trained with `shuffle=False`, seeds fixed to 42, early stopping on `val_loss` (patience 6, restore best weights) in phase A; phase B refits on `train+val` for the best epoch count. Epochs actually used: GRU — Bitcoin 1, Gold 8, Silver 9; LSTM — Bitcoin 2, Gold 1, Silver 4.

Why we used it. To test whether sequence models capture temporal structure that tabular models cannot (a standard FYP expectation).

Strengths	Limitations	Role in our project
Can model order and long-range dependence in principle.	Needs far more data than ~2,000 samples; stopping after 1-9 epochs shows they learn little beyond the mean.	Members of the Combined forecast (each contributes 1/6). Best single model by validation for Silver: GRU.

Viva-critical observation. Several final models stopped extremely early (LightGBM 10 trees for Gold and Silver; GRU 1 epoch for Bitcoin; LSTM 1 epoch for Gold). Early stopping found that continuing to fit only made validation error worse — i.e. the models are close to “predict the average return”. This is consistent with the predicted-return standard deviation of ~0.1 % versus 1.7–3.2 % for actual returns on the test set.

10.8 The stacked ensemble (experiment, not served)

`src/models/ensemble_model.py::fit_stack`: the out-of-fold predictions of Ridge, LightGBM, CatBoost and GRU from the 4 walk-forward folds are used as inputs to a **non-negative Ridge meta-model** (`Ridge(alpha=1.0, positive=True)`): $\hat{f}_{\text{stack}} = w_0 + \sum w_k \hat{f}_k$, $w_k \geq 0$. Learned weights (`results/stacking/*.json`):

Asset	Ridge	LightGBM	CatBoost	GRU	Intercept (std. units)
Bitcoin	0.000	0.387	0.309	0.000	0.0064
Gold	0.084	0.000	1.177	0.593	0.0833
Silver	0.000	0.546	0.000	0.989	0.0205

The stack is reported in the test tables (“Stacked”) but it is not a member of the served Combined forecast because it is itself a blend of some of these models. Its weights vary widely across assets — a symptom that the base models carry no stable signal.

11. Model-by-Model Mathematical Explanation

11.1 Common set-up

Let $x_t \in \mathbb{R}^f$ be the scaled feature vector of day t and $y_t = (\ln(P_{t+1}/P_t) - \mu)/\sigma$ the standardised target (μ, σ from the training set). Every model learns a function f with $\hat{y}_t = f(x_t)$ (tabular) or $\hat{y}_t = f(x_{t-29}, \dots, x_t)$ (recurrent). The prediction is converted back: $\hat{r}_t = \sigma \cdot \hat{y}_t + \mu$ and $P_{t+1} = P_t \cdot \exp(\hat{r}_t)$.

11.2 Ridge

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$
$$\text{minimise } \sum_i (y_i - \hat{y}_i)^2 + \alpha \cdot \sum_j \beta_j^2 \quad \alpha = 100 \text{ in our final models}$$

β_0 is the intercept; β_j is how much the prediction changes per unit of feature j ; α controls the penalty. Large α shrinks all β_j towards 0 and the prediction towards the mean — exactly what happens here.

11.3 Random Forest

- A **decision tree** splits the data by questions like “is ewma_vol < 0.2?” until it reaches leaves (here depth ≤ 3 , ≥ 30 samples per leaf); a leaf predicts the average target of its samples.
- **Bootstrapping**: each of the 300 trees is trained on a random sample of rows drawn with replacement.
- **Random feature selection**: at each split only a random subset of features is considered, which decorrelates the trees.
- **Aggregation**: $\hat{y} = (1/300) \sum \text{tree}_k(x)$. Averaging many noisy but different trees lowers variance.

11.4 LightGBM and CatBoost (gradient boosting)

$$F_0(x) = \text{mean}(y)$$
$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x) \quad h_m \text{ fits the residuals (negative gradient)} \ y_i - F_{m-1}(x_i)$$
$$\hat{y} = F_M(x)$$

- **Boosting / sequential trees**: each new small tree h_m is fitted to the current errors (residuals) of the sum of all previous trees.
- **Learning rate η** : how much of each tree is added (0.01–0.03 here) — small values need more trees but generalise better.
- **Tree depth / leaves**: num_leaves (LightGBM 8–31) or depth (CatBoost 3–7) limit complexity.
- **Regularisation**: reg_lambda / l2_leaf_reg penalise large leaf values; min_child_samples forbids tiny leaves.
- **Early stopping**: the number of trees M is chosen where validation error stops improving (here 10–606).
- LightGBM grows trees leaf-wise; CatBoost uses symmetric (oblivious) trees.

11.5 GRU and LSTM

Both read the window $x_1 \dots x_{30}$ in order and maintain a hidden state h_t (the memory). Gates (values in 0–1, computed from the current input and the previous hidden state) decide what to keep or forget.

LSTM (gates: forget f , input i , output o ; cell state c):
 $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$ $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$
 $c_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$ $c_t = f_t \odot c_{t-1} + i_t \odot c_t$ $h_t = o_t \odot \tanh(c_t)$
GRU (gates: update z , reset r ; no separate cell state):
 $z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$ $r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$
 $\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t])$ $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$
Our head: $h_{30} \rightarrow \text{Dropout}(p) \rightarrow \text{Dense}(16, \text{ReLU}) \rightarrow \text{Dense}(1) = \hat{y}$ loss = mean($(y - \hat{y})^2$), optimiser Adam(lr = 0.001)

Why LSTM/GRU for time series: the hidden state lets the network carry information about earlier days to later days and the gates mitigate vanishing gradients, so patterns spread over several days can in principle be learned. GRU is the lighter variant (2 gates, ~25 % fewer parameters), which is why we compare both on identical settings. **Dropout** randomly switches off a fraction (0.2 or 0.4) of the units during training to reduce over-fitting.

11.6 Evaluation mathematics used later

$$\text{RMSE} = \sqrt{\text{mean}(r - \hat{r})^2}$$
$$\text{MAE} = \text{mean}|r - \hat{r}|$$
$$R^2 = 1 - \frac{\sum(r - \hat{r})^2}{\sum(r - \bar{r})^2}$$
$$\text{MAPE} = \text{mean}|P - \hat{P}| / P \times 100$$
$$\text{Diebold-Mariano: } d_t = e^2_{\text{model}} - e^2_{\text{naive}}; \text{ DM} = d^* / \sqrt{(\text{Var}(d)/n)} \quad (\text{Harvey small-sample correction})$$

12. Hyper-parameters

12.1 How they were chosen

- **Searched (grid search) with cross-validation** — not hand-picked and not Bayesian:
src/training/tune_models.py evaluates every configuration of a small grid with 4-fold expanding-window walk-forward validation on train+val and keeps the one with the lowest mean fold RMSE in return space. The test set is never read.
- Per-configuration fold scores are logged in results/tuning/<asset>_<model>.csv; winners in results/tuning/best_params.json, read by config.get_params() at training time.
- Defaults in config.DEFAULT_PARAMS are only starting points/fallbacks.
- Grids are deliberately small (the dataset is small): Ridge 4 configs, Random Forest 4, LightGBM 48, CatBoost 12, GRU 2, LSTM 2.

Model	Grid searched (tune_models.py::GRIDS)
Ridge	alpha $\in \{0.1, 1, 10, 100\}$
RandomForest	n_estimators = 300; max_depth $\in \{3, 6\}$; min_samples_leaf $\in \{10, 30\}$
LightGBM	n_estimators = 600; learning_rate $\in \{0.01, 0.03\}$; num_leaves $\in \{4, 8, 15, 31\}$; min_child_samples $\in \{20, 50, 100\}$; subsample = 0.8; colsample_bytree = 0.8; reg_lambda $\in \{0, 5\}$
CatBoost	iterations = 800; learning_rate $\in \{0.01, 0.03\}$; depth $\in \{3, 5, 7\}$; l2_leaf_reg $\in \{3, 10\}$
GRU / LSTM	units = 32; dropout $\in \{0.2, 0.4\}$; learning_rate = 0.001; batch_size = 64; epochs = 40; patience = 6

12.2 Final values (results/tuning/best_params.json) and learned stopping points

Model	Hyper-parameter	Bitcoin	Gold	Silver	Purpose
Ridge	alpha	100	100	100	L2 penalty strength (shrinks weights)
RandomForest	n_estimators / max_depth / min_samples_leaf	300 / 3 / 30	300 / 3 / 30	300 / 3 / 30	number of trees / depth limit / minimum leaf size
LightGBM	learning_rate	0.03	0.01	0.01	step size of each tree
	num_leaves	31	31	8	tree complexity
	min_child_samples	20	100	50	minimum samples per leaf
	reg_lambda	0.0	5.0	5.0	L2 regularisation of leaf values
	subsample / colsample_bytree	0.8 / 0.8	0.8 / 0.8	0.8 / 0.8	row / column sampling
	n_estimators used (early stopping)	81 (of 600)	10 (of 600)	10 (of 600)	trees actually kept
CatBoost	learning_rate / depth / l2_leaf_reg	0.03 / 7 / 3.0	0.01 / 3 / 10.0	0.01 / 3 / 10.0	step / tree depth / L2
	iterations used (early stopping)	50 (of 800)	606 (of 800)	82 (of 800)	trees actually kept
GRU	units / dropout / lr / batch / patience	32 / 0.4 / 0.001 / 64 / 6	32 / 0.4 / 0.001 / 64 / 6	32 / 0.4 / 0.001 / 64 / 6	hidden size / regularisation / learning rate / batch / early-stop patience
	epochs used	1	8	9	epochs kept (max 40)
LSTM	units / dropout / lr / batch / patience	32 / 0.4 / 0.001 / 64 / 6	32 / 0.2 / 0.001 / 64 / 6	32 / 0.2 / 0.001 / 64 / 6	same as GRU
	epochs used	2	1	4	epochs kept (max 40)

Other fixed settings: random_state = 42 (Random Forest, LightGBM), random_seed = 42 (CatBoost), set_all_seeds(42) for Keras, SEQ_LEN = 30, CV_FOLDS = 4. Observation: Ridge's α sits at the upper edge of its grid and Random Forest's depth at the lower edge — the search keeps choosing the most conservative option.

13. Model Training Pipeline

What happens when you run `make preprocess tune train stack evaluate` (verified step by step against the code):

#	Step	File → function	Purpose
1	Load raw CSVs	<code>preprocessing.py</code> → <code>DataCleaner.load_data</code>	read <code>data/raw/<asset>_data.csv</code> , parse timestamps
2	Clean	<code>DataCleaner.clean_data</code>	de-duplicate, sort, drop non-positive prices, crypto calendar fill, log return
3	Generate features	<code>add_moving_averages</code> , <code>add_rsi</code> , <code>add_macd</code> , <code>add_advanced_ta</code> , <code>add_relative_features</code> , <code>add_lag_returns</code> , <code>add_rolling_volatility</code> , <code>add_calendar_features/lag_post_settlement_features</code> , <code>add_external_features</code>	build stationary indicators; align macro by the close-time rule
4	Drop invalid rows & validate	<code>dropna</code> , <code>validate_features</code>	remove warm-up rows; fail on NaN/inf/<100 rows
5	Split chronologically	<code>normalize_and_split</code>	train ≤ 2025-09-10, val ≤ 2026-02-17, test = rest
6	Fit scaler on train, transform all	<code>normalize_and_split</code>	MinMax (0,1) fitted on train rows only
7	Save	<code>save_data</code>	<code>*_features.csv</code> , <code>*_{train,val,test}_scaled.csv</code> , <code><prefix>_scaler.pkl</code>
8	Build supervised dataset	<code>build_dataset</code>	windows, target (next-day log return), standardise target with train stats, split by target dates
9	Tune	<code>tune_models.py</code> → <code>tune</code> , <code>cv_evaluate</code>	grid search with 4 walk-forward folds → <code>best_params.json</code>
10	Train phase A	<code>train_models.py</code> → <code>train_asset</code>	fit on train, monitor val; save train/val metrics and loss curves; determine epochs/trees
11	Train phase B	<code>train_asset</code> (final=True)	refit on train+val with the fixed stopping point → deployed model
12	Save models/feature importance	<code>BaseModel.save</code> , <code>feature_importance</code>	<code>data/models/*.pkl .cbm .keras</code> (+ JSON side-cars); <code>results/feature_importance.csv</code>
13	Stacking experiment	<code>ensemble_model.py</code> → <code>fit_stack</code>	meta-model on out-of-fold predictions → <code><prefix>_stack.pkl</code>
14	Evaluate once on test	<code>backtesting.py</code> → <code>main</code>	CV table, test predictions of every model + Combined, band calibration, <code>model_status.json</code>
15	Figures / experiments	<code>plots.py</code> , <code>run_experiments.py</code> , <code>before_after.py</code>	report figures; design experiments on validation only
16	Load during prediction	<code>prediction.py</code> → <code>_window</code> , <code>_load</code>	reload scaler, models, target stats from <code>model_status.json</code>

13.1 Two-phase training explained

Models that need a stopping point (number of trees/epochs) face a trap: if the same validation data picks the stopping point *and* is used to report performance, the report is optimistic. The code therefore trains twice: **Phase A** — fit on train, early-stop on validation, and report honest train/validation metrics; **Phase B** — refit on train+val for exactly the stopping point found in A. Phase B's model is saved and used for the test set and for predictions. (The docstring notes this fixed an older CatBoost bug where validation leaked into its own monitor.)

13.2 Train vs validation errors — why they differ

Validation RMSE (return) is much larger than training RMSE for the metals (Gold 0.0097 train vs 0.0203 val; Silver 0.019 train vs 0.055 val). This is **not classic over-fitting**: it reflects a far more volatile validation period (late 2025–early 2026). The target is z-scored with training statistics, which is why GRU validation loss in the Gold loss curve is ≈ 4.3 in standardised units. The random-walk baseline shows the same jump (Silver validation RMSE 0.0554), so the models are not worse than the baseline there.

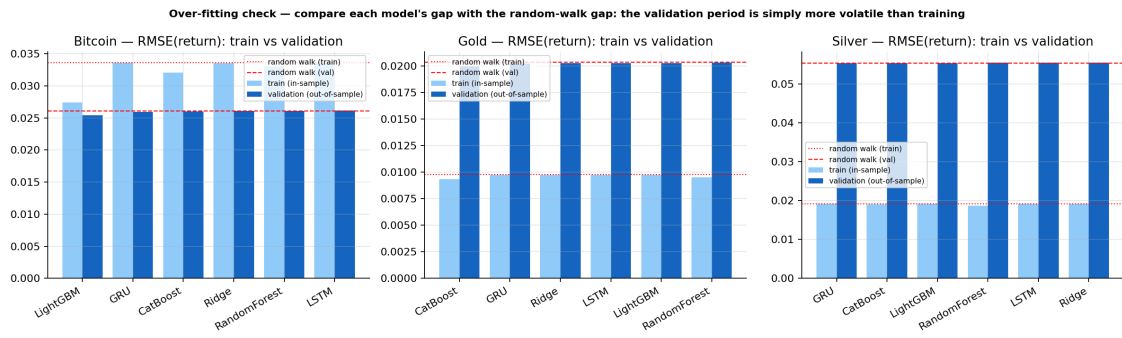


Figure 13.1 — Train vs validation RMSE per model (results/figures/overfitting_gap.png).

14. Model Evaluation

All metrics live in `src/utils/metrics.py` (`evaluate_forecast`) and are computed identically for every model on every split. Two “spaces” are reported: **return space** (what the models predict) and **USD space** (what a user sees).

Metric (key)	Measures	Better	Why useful / how to read
RMSE_ret	root-mean-square error of the log return	lower	Main model-comparison metric (tuning and selection use it). Punishes big misses.
MAE_ret	mean absolute error of the log return	lower	Typical size of a miss; less sensitive to outliers than RMSE.
R2_ret	1 – SSE/SST on returns	higher (0 = predicting the mean, <0 = worse)	The honest “skill” number. ≈ 0 here for every model.
MAE_usd, RMSE_usd	errors of the reconstructed price $P_t \cdot \exp(\hat{r})$	lower	What a user feels in dollars; but the naive forecast scores almost the same.
MAPE_usd	mean absolute % price error	lower	Looks small (1.3–2.5 %) for any forecast near today’s price — not proof of skill.
DirAcc_pct (+ n, p-value)	% of non-flat days with $\text{sign}(\hat{r}) = \text{sign}(r)$; one-sided binomial test vs 50 %	higher (50 % = coin flip)	Direction hit-rate. A value is only meaningful if $p < 0.05$.
UpCalls_pct / UpDays_pct	share of days the model said UP / share of days that were up	—	Added in the final audit: a model that says UP almost always scores the share of up days by construction (Gold 97 % UP calls).
DM_pvalue (vs naive)	Diebold-Mariano test of squared-error difference vs random walk	$p < 0.05$ = significantly different	Tells whether a small RMSE gap is real or luck. All our $p \geq 0.17$.
pred_std / true_std	std of predicted vs actual returns	—	Shows how “flat” the forecast is (0.1 % vs 1.7–3.2 %).
strategy_return_pct, buy_hold_return_pct, Sharpe, n_trades	long/flat backtest (hold iff $\hat{r} > 0$), 10 bps cost per position change	higher	Sanity check only; negative for the served forecast in all three assets.
coverage_pct, QLIKE (band)	share of days $ r - \hat{r} \leq \sigma_t$ (nominal 68.3 %); QLIKE of σ_t^2	coverage ≈ 68 ; QLIKE lower	Tests the $\pm 1\sigma$ uncertainty band.

14.1 Headline: served Combined forecast vs random walk (untouched test set)

Asset	Test days	Forecast	MAE (\$)	RMSE (\$)	MAPE %	RMSE (ret)	R ² (ret)	Dir. acc % (p)	UP calls / up days	DM p
Bitcoin	207	Combined	1,063.94	1,447.37	1.52	0.02072	+0.006	46.9 (0.83)	37% / 49%	0.49
		Random walk	1,066.42	1,451.73	1.52	0.02079	-0.001	—	—	—
Gold	143	Combined	58.20	75.73	1.29	0.01671	-0.001	50.3 (0.50)	97% / 50%	0.97
		Random walk	58.26	75.76	1.29	0.01671	-0.002	—	—	—
Silver	143	Combined	1.78	2.37	2.47	0.03180	+0.005	53.8 (0.20)	91% / 52%	0.46
		Random walk	1.79	2.37	2.49	0.03189	-0.001	—	—	—

What our results mean. (1) Dollar errors of Combined and the random walk are practically identical (within ± 0.4 %). (2) R^2 of the return ≈ 0 (–0.001 to +0.006): the models explain none of tomorrow’s return variation. (3) Direction accuracy 46.9 / 50.3 / 53.8 % with $p = 0.83$ / 0.50 / 0.20: indistinguishable from a coin flip. (4) For Gold and Silver the “UP” calls are 97 % and 91 % of days, so direction is mostly the assets’ average drift, not timing. (5) DM $p = 0.49$ / 0.97 / 0.46: the tiny RMSE improvements are not statistically meaningful.

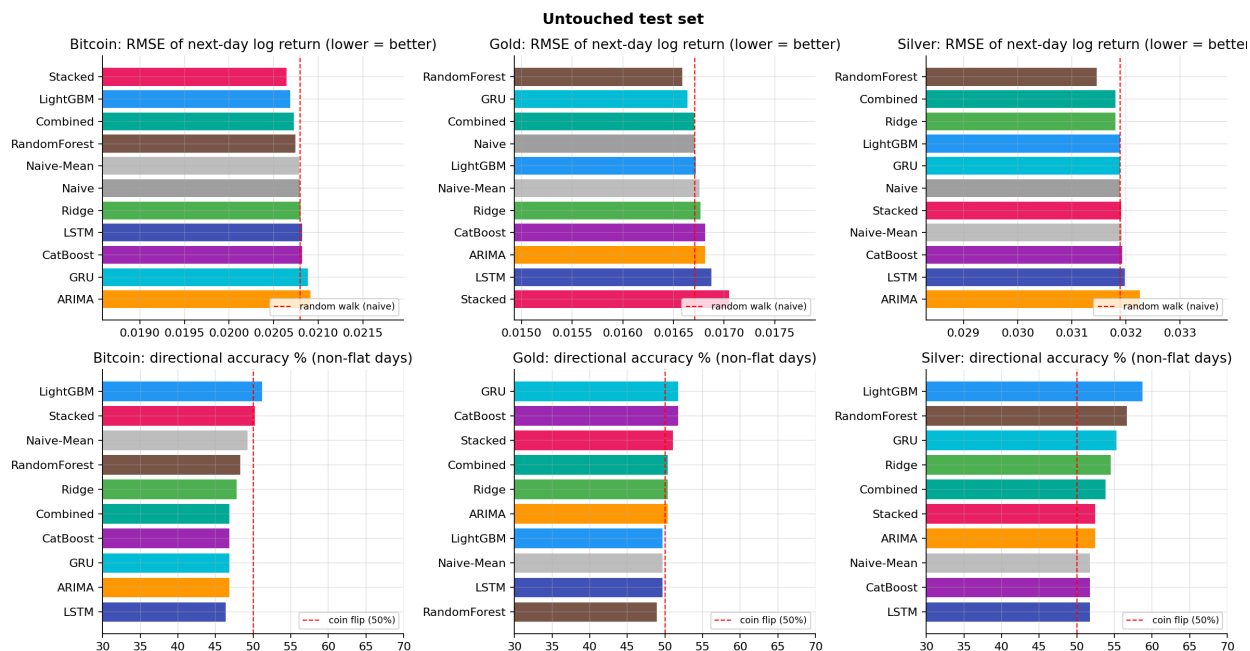


Figure 14.1 — All models vs the random walk on the test set (results/figures/model_comparison_test.png).

15. Baseline Comparison

A baseline **is implemented** and is central to the project. Three are reported in every table:

- **Naive (random walk):** $P_{t+1} = P_t$, i.e. $\hat{r} = 0$ — the main benchmark.
- **Naive-Mean:** $\hat{r} = \text{mean training return}$.
- **ARIMA(p,0,q):** classical statistical forecast, order by AIC.

Asset	Naive RMSE (ret) test	Combined	Best single (test)	Naive-Mean	ARIMA
Bitcoin	0.02079	0.02072	LightGBM 0.02068	0.02078	0.02091
Gold	0.01671	0.01671	RandomForest 0.01659	0.01675	0.01681
Silver	0.03189	0.03180	RandomForest 0.03146	0.03192	0.03226

Conclusion: **no model beats the baseline significantly**. Individual models occasionally edge it by chance (e.g. Random Forest on Silver: RMSE 0.03146 vs 0.03189, DM $p = 0.17$; LightGBM Silver direction 58.7 %, $p = 0.02$ — a single significant result among ~30 model-asset tests, i.e. what chance alone produces; the picked-in-hindsight model would not have been chosen by the validation rule). ARIMA has the highest test RMSE (return) of all models for Bitcoin and Silver.

A baseline already exists, so no “recommended addition” is needed. The one thing that would strengthen credibility further would be a multiple-testing note (see Chapter 39).

16. Ensemble / Combined Prediction

The served forecast is the **Combined** forecast, defined in `src/inference/prediction.py` (`COMBINED`, `_predict_from_window`) and mirrored in `backtesting.py::test_predictions` so that the evaluation scores exactly what the dashboard serves.

```
r^_Combined = (1/6) * ( r^_Ridge + r^_RandomForest + r^_LightGBM + r^_CatBoost + r^_GRU + r^_LSTM )
P^{t+1} = P_t * exp( r^_Combined )          direction = UP if r^_Combined > 0 else DOWN
```

Question	Answer (from code)
Which models contribute?	All trained base models that have a saved artefact: Ridge, RandomForest, LightGBM, CatBoost, GRU, LSTM (<code>combined_members</code>). The stacked model and baselines are not members.
Why combine?	Different models make different errors; averaging reduces model-specific variance. With no model clearly better, a plain average is the robust choice.
Equal or weighted?	Equal weights (1/6 each) . No weights are fitted, so nothing is selected on the test set.
Are weights learned?	No (the stacked experiment does learn weights, but it is not served).
Is the ensemble better than individual models?	No single winner exists. On the walk-forward folds Combined RMSE is 0.03159 (Bitcoin), 0.01113 (Gold), 0.02390 (Silver) vs naive 0.03160 / 0.01114 / 0.02391. On the test set it is 0.3 %, 0 % and 0.3 % below the naive RMSE (not significant). It is not the best number in any table — it is the least “lucky” choice.
How is the final prediction calculated?	Each model predicts a standardised return → converted with $\hat{y} \cdot sd + \mu$ → averaged in return space → exponentiated onto the last close.

16.1 Why an ensemble can reduce error — and its limits

If two forecasts have errors that are not perfectly correlated, the error of their average is smaller than the average of their errors (variance reduction). Averaging 6 models whose predictions differ by hundredths of a percent, however, can only produce a forecast of similar magnitude — the individual models’ predictions in our real run (Chapter 27) range from +0.02 % to +0.29 % for Bitcoin. **Limits:** (1) it cannot create signal that no member has; (2) the members are correlated (same features); (3) averaging pulls predictions towards zero, which is why the Combined forecast’s direction calls are sometimes (Bitcoin test: 37 % UP calls) driven by small offsets.

Implementation caveat to know: `combined_members` includes only models whose artefact file exists. If one model file were missing, the code would silently average the remaining models (the dashboard text would then say “all N trained models”). The tests check the normal six-model case.

16.2 The stacked-ensemble comparison

For completeness the stacked meta-model was scored on the test set: Bitcoin RMSE (ret) 0.02065 (best of all, DM $p = 0.44$), Gold 0.01705 (worst), Silver 0.03191. It is the best Bitcoin number in the table yet not significant, and is the worst for Gold — typical noise, which is why it was not promoted to the served forecast.

17. Final Prediction Flow

17.1 What happens when the user presses “Run prediction”

1. **User selects an asset** in the sidebar (`st.sidebar.selectbox`) and presses **Run prediction** (sets `st.session_state["run_prediction"] = True`). Deep link: `?asset=Gold&run=1`.
2. **Freshness check** (`streamlit_app.py`): if `live_data_is_current(asset)` (live file ends on the newest complete bar) or the live file is < 30 minutes old, no download; otherwise `update_live_data(asset, refresh_external=True)` runs.
3. **Live data refresh** (`sync_live_data.update_live_data`): download macro series and the asset (and gold for Silver) into `data/raw_live/`, run `DataCleaner.clean_data`, atomically write `data/processed/<prefix>_live_features.csv`.
4. `predict_next_day(asset, "Combined")` (`prediction.py`): load `features` (live file if present, else frozen file).
5. `_window`: select model-feature columns (all except `LEVEL_COLUMNS`), check the count equals the scaler's `n_features_in_`, transform with the saved scaler, take the last 30 rows → `Xseq (1,30,F)` and `Xt (1,F)`; also `P_t` and the date `t`.
6. **Each model predicts**: `_predict_one` loads the model from `data/models/` and computes $r = \text{predict}(...) \cdot \text{target_std} + \text{target_mean}$ (target stats from `model_status.json`); raises if non-finite.
7. **Combination**: $r^{\wedge} = \text{mean}(\text{member } r)$; $P^{\wedge} = P_t \cdot \exp(r^{\wedge})$.
8. **UP/DOWN**: $r^{\wedge} > 0 \rightarrow \text{"UP"}$ else `"DOWN"`.
9. **Band**: `_band` → $[P_t \cdot \exp(\hat{r} - \sigma_t), P_t \cdot \exp(\hat{r} + \sigma_t)]$, σ_t = unscaled `ewma_vol` on day `t`.
10. **Display**: final card (price, UP/DOWN, % change), band caption with its measured test coverage, table of every member's prediction, 60-day chart with the forecast, reliability table from `final_test_results.csv`.

17.2 Failure behaviour (verified in code and tests)

Situation	What happens	Where
Data cannot be downloaded / Yahoo empty or rate-limited	<code>fetch_asset</code> prints a warning and leaves the existing CSV untouched; <code>update_live_data</code> returns False (never raises). Dashboard keeps using the stored features; the “Sync” button shows “Yahoo Finance did not return data (rate limit). Using the stored dataset.”	<code>data_collection.py</code> , <code>sync_live_data.py</code> , <code>streamlit_app.py</code> ; <code>test_live_sync_survives_network_failure</code>
Yahoo has not published the newest bar yet (e.g. Bitcoin after 00:00 UTC)	Warning: forecast starts from the last complete bar available; press again later.	<code>streamlit_app.py</code> (warning using <code>expected_last_complete_bar</code>)
Downloaded data is corrupt (NaN/inf/too short)	<code>validate_features</code> raises <code>ValueError</code> ; <code>update_live_data</code> catches it and does not overwrite the live file.	<code>preprocessing.validate_features</code> ; <code>test_corrupt_live_data_raises_instead_of_returning_nan</code>
Processed data missing	Dashboard: “No processed data found. Run make preprocess first.” <code>load_features</code> raises <code>FileNotFoundError</code> .	<code>streamlit_app.py</code> , <code>prediction.load_features</code>
Model file missing	<code>combined_members</code> lists only existing artefacts; if none: <code>FileNotFoundError("No trained models found ... Run make train")</code> . A single missing member is silently skipped (caveat above). API returns HTTP 503.	<code>prediction.py</code> , <code>api/app.py</code>
<code>model_status.json</code> lacks target stats	Explicit <code>FileNotFoundError</code> telling you to run <code>make evaluate</code> (no silent wrong scale).	<code>prediction._target_stats</code> ; <code>test_missing_evaluation_is_reported_clearly</code>
Feature count mismatch with scaler	<code>ValueError("Feature mismatch ...")</code> .	<code>prediction._window</code>
Not enough history (< 30 rows)	<code>ValueError("Not enough history ...")</code> .	<code>prediction._window</code>
Model returns NaN/inf	<code>ValueError("... non-finite prediction ...")</code> .	<code>prediction._predict_one</code>
Any exception in the dashboard	Caught: <code>st.error("Prediction failed: ...")</code> and logged.	<code>streamlit_app.py</code>
API: unknown asset / model	HTTP 404; missing artefacts 503; other errors 500.	<code>api/app.py</code>
Invalid user input	The UI only offers fixed choices (asset select box, date select box limited to test days), so free-text invalid input is not possible.	Streamlit widgets

18. UP / DOWN Logic

```
direction = "UP"    if r^_Combined > 0
direction = "DOWN" otherwise      # prediction.py: ``UP' if r_hat > 0 else 'DOWN'``
```

- **Current price:** $P_t = \text{hist}["price"].\text{iloc}[-1]$, the last *complete* daily close in the features file.
- **Predicted price:** $\hat{P} = P_t \cdot \exp(\hat{r})$. Because $\exp(\hat{r}) > 1 \Leftrightarrow \hat{r} > 0$, comparing $\hat{P}$ with P_t is the same as testing the sign of $\hat{r}$.
- **Threshold:** none. Any positive $\hat{r}$, even +0.02 %, is UP. A forecast of exactly 0 would display DOWN (edge case; practically never happens).
- **Each member** gets its own UP/DOWN by the same rule; the dashboard shows “n of 6 models say UP”.
- **In evaluation** the sign of the predicted return is compared with the sign of the actual return on non-flat days (flat days are excluded).

UP/DOWN is only the sign of a tiny predicted return (e.g. +0.16 %) that is far smaller than the typical daily move (± 2 %). Our test results show the direction is correct about as often as a coin flip. **It is not a trading signal and it does not imply profit.** For Gold the combined forecast said UP on 97 % of the test days: its direction mostly reflects the long-run average drift.

19. Streamlit Dashboard

Entry file: app/streamlit_app.py (run with streamlit run app/streamlit_app.py / make serve; configuration entry “streamlit” in .claude/launch.json uses port 8501). It is a single-file, multi-tab app. Everything shown is read from project artefacts or computed live by prediction.py; nothing is hard-coded.

19.1 What the user sees, in order

1. **Page set-up:** dark theme CSS, wide layout, title “<Asset> — Next-Day Price Forecast”.
2. **Sidebar:** (a) **Asset** select box (Bitcoin / Gold / Silver); (b) **Run prediction** primary button with a caption “Runs all 6 trained models ... No model to choose”; (c) **Chart overlays** multiselect — Bollinger Bands, RSI, MACD (+ Volume for Bitcoin only); (d) **Sync live market data** button; (e) a caption block: served forecast, best single model by validation, horizon, data source and last complete bar date.
3. **Overview strip** (an HTML box): what is predicted, the served forecast and its reliability on the unseen days — and the plain statement “The forecast does not beat the random walk (Diebold–Mariano $p = \dots$)”.
4. **Five tabs:**

Tab	Contents
1 Forecast	Latest market data metrics (last close, 30-day high/low, 30-day volatility); after pressing Run: the final prediction card (predicted close, UP ▲ / DOWN ▼, % and \$ change), the $\pm 1\sigma$ band caption with measured coverage, a table of each model's prediction , a 60-day chart with the forecast and band, a “How reliable is this forecast?” table vs the random walk; below: price history with train/validation/test shading and optional indicator panels.
2 Predict a Day (unseen test)	Pick any test-period day; predict_for_date uses only data up to that day and reveals the actual next close, the error, hit/fail of the direction, whether the actual fell inside the band, a chart of the whole test period, and a per-model table.
3 Test-Set History	Every unseen test day: predicted vs actual price, error, random-walk error, direction HIT/FAIL, band and in-band flag; summary table; CSV download.
4 Model Performance	Test table for all models (Combined marked “served”), an “Honest reading” text built from the numbers, comparison bar charts, walk-forward table, predicted-vs-actual returns (line + scatter), feature-importance chart of a chosen member model, band calibration table, regime table, design experiments (E1–E4), training diagnostics (over-fitting gap, loss curves).
5 Methodology	Target, data, split, features (asset-specific), models, how the combination works, band, metrics; a details table.

19.2 Messages, warnings and errors

- `st.info("No evaluation found. Run make evaluate ...")`, `st.info("No processed data found. Run make preprocess first.")`, `st.warning("Run make evaluate first ...")` — missing artefacts.
- `st.warning(...)` when Yahoo has not yet published the newest bar; `st.sidebar.warning(...)` when sync fails.
- `st.error("Prediction failed: ...")` for any exception during prediction (also logged).
- Informational captions explain reading: “The UP/DOWN call is mostly the asset’s average drift ...”, “A predicted-price line always hugs the actual line ...”.
- Footer on every page: the disclaimer string (`prediction.DISCLAIMER`).

Headless smoke test run for this guide: all five tabs rendered for Bitcoin and Silver with zero exceptions (Streamlit AppTest). **Caching:** `@st.cache_data(ttl=300)` on CSV loaders; the live download is throttled to once per 30 minutes.

FastAPI companion (`src/api/app.py`, `make api`): `GET /health`, `GET /models`, `GET /predict/{bitcoin|gold|silver}[?model=GRU]`; response model `PredictionResponse` (price, return, direction, members, band, test metrics, disclaimer); CORS open to all origins.

20. Complete VS Code Project Structure

This is the actual directory of the repository (inspected on disk). Folders marked $\diamond$ exist locally but are git-ignored (.gitignore); they are created by the pipeline.

```
crypto-forex-prediction-system/
├── config.py                central configuration (paths, assets, split dates, target, feature policy, hyper-parameter defaults)
├── requirements.txt         direct dependencies (pinned core libraries)
├── requirements-lock.txt    full frozen environment (pip freeze)
├── Makefile                one-word commands: preprocess, tune, train, evaluate, serve, api, test, pipeline ...
├── .env.example            optional settings template (only TF_CPP_MIN_LOG_LEVEL; no secrets)
├── .gitignore              README.md  CHANGELOG.md
├── app/
│   └── streamlit_app.py    the Streamlit dashboard (5 tabs)
├── src/
│   ├── README.md          package map
│   ├── data/
│   │   ├── data_collection.py  Yahoo OHLCV download (complete bars only)
│   │   ├── external_data.py    macro series + Fear & Greed download
│   │   ├── market_calendar.py  complete-bar rule, next trading day
│   │   ├── preprocessing.py    DataCleaner (features, split, scaler) + build_dataset (target, windows)
│   │   ├── eda.py             ADF tests, ACF/PACF, distributions
│   │   └── sync_live_data.py    live refresh for the dashboard (data/raw_live)
│   ├── models/
│   │   ├── registry.py        uniform model interface for all 9 models + save/load
│   │   ├── model_gru.py       Keras GRU builder
│   │   ├── model_lstm.py      Keras LSTM builder
│   │   └── ensemble_model.py   stacked ensemble (experiment) + StackedModel wrapper
│   ├── training/
│   │   ├── tune_models.py     walk-forward grid search (train+val)
│   │   └── train_models.py     two-phase final training
│   ├── evaluation/
│   │   ├── cross_validation.py expanding-window folds, cv_evaluate
│   │   ├── backtesting.py     THE single test-set evaluation + result tables + model_status.json
│   │   └── plots.py           report figures
│   ├── experiments/
│   │   ├── run_experiments.py  E1 data size, E2 ablation, E3 horizon, E4 volatility
│   │   └── before_after.py     3-year system vs current system on identical days
│   ├── inference/
│   │   └── prediction.py       predict_next_day, predict_for_date, combined forecast, band
│   ├── api/
│   │   └── app.py             FastAPI service
│   ├── utils/
│   │   ├── metrics.py         RMSE/MAE/MAPE/R2, directional accuracy, Diebold-Mariano, backtest, band calibration
│   │   ├── inverse_transform.py  $P^* = P.t.exp(r^*)$ 
│   │   ├── reproducibility.py  set_all_seeds
│   │   └── logging_config.py   logging setup
│   └── (each package folder also contains an __init__.py)
├── scripts/
│   └── retrain.py             run the whole pipeline as one command
├── tests/
│   └── 43 pytest cases (conftest.py + 5 test files)
├── notebooks/
│   └── 4 companion notebooks (load and display script outputs)
├── data/
│   ├── (mostly git-ignored)
│   ├── raw/  $\diamond$           frozen downloads used for ALL reported results (9 CSVs)
│   ├── raw_3y_backup/       older 3-year raw files (tracked in git)
│   ├── raw_live/  $\diamond$      dashboard refreshes (never touches raw/)
│   ├── processed/  $\diamond$    * features.csv, *_{train,val,test}_scaled.csv, *_live_features.csv
│   └── models/              18 trained models, 3 scalers, 3 stack files; model_status.json is tracked in git
├── results/
│   ├── tuning/              best_params.json + per-model grid logs
│   ├── stacking/            stack weights
│   ├── histories/           GRU/LSTM loss curves (JSON)
│   ├── predictions/         test-period prediction history per asset
│   ├── experiments/         E1-E4 + before_after + summary
│   ├── figures/             24 PNG figures
│   └── archive_3y_final/, archive_sameday_macro_leak/, archive_pre_fix/  earlier results kept for the audit trail
├── docs/FYP_Technical_Report.pdf  the earlier complete technical report
├── .github/workflows/ci.yml        CI: install, import checks, pytest (Python 3.11)
├── .claude/launch.json             editor/tool settings
├── .vscode/settings.json           editor/tool settings
├── venv/  $\diamond$  catboost_info/  $\diamond$  .pytest_cache/  $\diamond$  (local environment and caches)
```

20.1 Top-level and configuration files

File	config.py
Purpose	Single source of truth for every “where is this decided?” question.
Contains / key items	Paths; ASSET_CONFIG (ticker, type, source, filename, prefix); DATA_START_DATE; TARGET_COL, PREDICTION_HORIZON_DAYS, TASKS; split dates TRAIN_END, VAL_END; LEVEL_COLUMNS; EXTERNAL_SAME_DAY, POST_SETTLEMENT_FEATURES; SEQ_LEN = 30; DEFAULT_PARAMS; CV_FOLDS = 4; MODEL_STATUS_PATH. Creates the data/results folders on import.
Important functions	get_prefix(asset), get_params(model, asset, task) (tuned params from best_params.json, else defaults), load_model_status()
Inputs	optional JSON files (best_params.json, model_status.json)
Outputs	constants and dictionaries

Used by	almost every module
Depends on	standard library only
Pipeline position	Used by all stages

File	Makefile
Purpose	Short commands for each pipeline stage.
Contains / key items	targets: install, test, collect-data, preprocess, eda, tune, train, stack, evaluate, figures, experiments, pipeline, predict, serve, api, clean
Important functions	—
Inputs	—
Outputs	—
Used by	the developer / examiner
Depends on	python (variable PY)
Pipeline position	Orchestration

File	requirements.txt / requirements-lock.txt
Purpose	Dependency list (direct, pinned core libraries) and a full frozen environment.
Contains / key items	see Chapter 23
Important functions	—
Inputs	—
Outputs	—
Used by	pip install -r, CI
Depends on	—
Pipeline position	Environment

File	.env.example
Purpose	Template for optional environment variables.
Contains / key items	only TF_CPP_MIN_LOG_LEVEL=3 (silences TensorFlow logs). No API keys exist.
Important functions	—
Inputs	—
Outputs	—
Used by	developer (copy to .env; .env is git-ignored)
Depends on	—
Pipeline position	Environment

File	.github/workflows/ci.yml
Purpose	Continuous integration on push/PR to main.
Contains / key items	Python 3.11; install requirements; import checks of config/preprocessing/registry/backtesting; pytest tests/ -q
Important functions	—
Inputs	—
Outputs	pass/fail
Used by	GitHub
Depends on	requirements.txt, tests/
Pipeline position	Quality gate

20.2 src/data — data layer

File	src/data/data_collection.py
Purpose	Download daily OHLCV for the three assets from Yahoo Finance.

Contains / key items	CLI with --start; writes timestamp, open, high, low, price, volume.
Important functions	fetch_asset(asset_name, start, out_dir) → DataFrame (empty on failure, existing CSV untouched); verify_data(df, asset) prints range/nulls
Inputs	asset name, start date
Outputs	data/raw/<asset>_data.csv (or raw_live/)
Used by	sync_live_data.py, Makefile collect-data, retrain.py
Depends on	config, market_calendar.drop_incomplete_bars, yfinance
Pipeline position	Stage 1 — data acquisition

File	src/data/external_data.py
Purpose	Download macro series and the Fear & Greed index.
Contains / key items	EXTERNAL_SOURCES (DXY, CrudeOil, VIX, TreasuryYields, SP500).
Important functions	fetch_yfinance_external(name, out_dir), fetch_fear_greed_index(out_dir), fetch_all_external_data(out_dir)
Inputs	tickers / alternative.me URL
Outputs	dxy, crude_oil, vix, tn timer, sp500, fear_greed CSVs
Used by	sync_live_data.py, Makefile, retrain.py
Depends on	config, market_calendar, yfinance, urllib
Pipeline position	Stage 1

File	src/data/market_calendar.py
Purpose	Complete-bar rule and trading-calendar helpers.
Contains / key items	ET zone; SESSION_END_ET = (17, 15).
Important functions	complete_bar_cutoff(asset_type, now), drop_incomplete_bars(df, asset_type), next_trading_day(asset_type, date), expected_last_complete_bar(asset_type)
Inputs	DataFrame / asset type / time
Outputs	filtered DataFrame, dates
Used by	data_collection, external_data, prediction, streamlit_app
Depends on	pandas, zoneinfo
Pipeline position	Stage 1 and prediction labels

File	src/data/preprocessing.py
Purpose	The core of the data layer: cleaning, all features, the frozen chronological split, the scaler, the target and the windows.
Contains / key items	class DataCleaner; functions create_sequences (used by tests; build_dataset uses _make_windows), _make_windows, _task_target, build_dataset, run_cleaning_pipeline.
Important functions	DataCleaner.load_data/clean_data/add_*/_align_external/normalize_and_split/save_data; build_dataset(asset, seq_len, task, train_start) returns a dict (X, Xt, y, y_real, dates, prev, true, naive, scaler, inv/fwd ...)
Inputs	raw CSVs (+ processed CSVs and scaler for build_dataset)
Outputs	features CSV, scaled CSVs, scaler.pkl; the dataset dict
Used by	tune, train, ensemble, backtesting, experiments, sync_live_data, tests
Depends on	config, pandas, numpy, scikit-learn, ta, joblib
Pipeline position	Stages 2-8 (cleaning → dataset)

File	src/data/eda.py
Purpose	Exploratory analysis for the report.
Contains / key items	ADF tests, ACF/PACF, distribution and volatility-clustering figures (training window only).
Important functions	adf(series), main()
Inputs	*_features.csv
Outputs	results/eda_summary.csv, results/figures/eda_*.png
Used by	Makefile eda, retrain.py
Depends on	statsmodels, scipy, matplotlib

Pipeline position	Analysis (not on the prediction path)
File	src/data/sync_live_data.py
Purpose	Refresh the latest market data for live prediction without touching frozen data.
Contains / key items	writes data/processed/<prefix>_live_features.csv atomically.
Important functions	update_live_data(asset, refresh_external=True) → bool (never raises on API failure)
Inputs	asset name
Outputs	live features file
Used by	streamlit_app.py
Depends on	DataCleaner, fetch_asset, fetch_all_external_data
Pipeline position	Dashboard refresh path

20.3 src/models — model layer

File	src/models/registry.py
Purpose	One uniform interface for all nine models; two-phase fitting; persistence.
Contains / key items	BaseModel; baselines NaiveModel, NaiveMean, ARIMAModel; RidgeModel, RandomForestModel, LightGBMModel, CatBoostModel; _KerasSequenceModel → GRUModel, LSTMModel; groups TABULAR, RECURRENT, BASELINES, ALL_MODELS.
Important functions	make_model(name, params), load_trained(name, asset), artefact_exists(name, asset), artefact_stem; per model fit(Xseq_tr, Xt_tr, y_tr, Xseq_val, Xt_val, y_val, final), predict(Xseq, Xt), save, load, feature_importance
Inputs	training arrays
Outputs	fitted model objects; files data/models/<prefix>_<model>.pkl .cbm .keras (+ .json)
Used by	train_models, cross_validation, backtesting, ensemble_model, prediction, experiments
Depends on	config, reproducibility, sklearn, lightgbm, catboost, tensorflow, statsmodels
Pipeline position	Stages 9-12, 16

File	src/models/model_gru.py, model_lstm.py
Purpose	Keras network builders.
Contains / key items	Input(seq_len, F) → GRU/LSTM(units) → Dropout → Dense(16, relu) → Dense(1); Adam, MSE.
Important functions	build_gru_model(seq_len, n_features, units, dropout, learning_rate), build_lstm_model(...)
Inputs	sequence length, feature count, hyper-parameters
Outputs	compiled Keras model
Used by	registry.GRUModel/LSTMModel
Depends on	tensorflow
Pipeline position	Stage 10-11

File	src/models/ensemble_model.py
Purpose	Stacked ensemble experiment and its inference wrapper.
Contains / key items	non-negative Ridge meta-model on out-of-fold predictions of Ridge, LightGBM, CatBoost, GRU.
Important functions	fit_stack(asset, bases), stack_path(asset), class StackedModel(asset).predict
Inputs	dataset dict, walk-forward folds
Outputs	data/models/<prefix>_stack.pkl, results/stacking/*_stack_weights.json
Used by	backtesting (Stacked row), prediction (available_models, "Stacked" option)
Depends on	build_dataset, cv_evaluate, registry
Pipeline position	Stage 13 (experiment)

20.4 src/training and src/evaluation

File	src/training/tune_models.py
-------------	-----------------------------

Purpose	Hyper-parameter search by walk-forward validation.
Contains / key items	GRIDS dictionary.
Important functions	grid(model), tune(asset, model, data), main()
Inputs	dataset dict (train+val only)
Outputs	results/tuning/best_params.json, <asset>_<model>.csv, tuning_summary.csv
Used by	Makefile tune, retrain.py; output read by config.get_params
Depends on	build_dataset, cv_evaluate
Pipeline position	Stage 9

File	src/training/train_models.py
Purpose	Train the deployed models in two phases.
Contains / key items	phase A (honest metrics, loss curves) and phase B (train+val refit).
Important functions	train_asset(asset, model_names, task), main()
Inputs	dataset dict, tuned params
Outputs	models in data/models/, results/train_val_metrics.csv, feature_importance.csv, histories/*.json
Used by	Makefile train, retrain.py
Depends on	build_dataset, make_model, evaluate_task, get_params
Pipeline position	Stages 10-12

File	src/evaluation/cross_validation.py
Purpose	Expanding-window walk-forward validation.
Contains / key items	never touches the test split.
Important functions	walk_forward_splits, trainval_arrays, cv_evaluate(model, params, data, n_splits, return_oof, feature_idx), primary_metric, summarise_folds
Inputs	model name, params, dataset dict
Outputs	per-fold metrics (and out-of-fold predictions)
Used by	tune_models, ensemble_model, backtesting, run_experiments
Depends on	registry.make_model, metrics.evaluate_task, sklearn TimeSeriesSplit
Pipeline position	Stage 9, 13, 14

File	src/evaluation/backtesting.py
Purpose	The only script that reads the test split. Final evaluation of every model and of the Combined forecast.
Contains / key items	ML_MODELS, BASELINE_MODELS, COMBINED.
Important functions	cv_table, combined_cv_row, test_predictions, band_rows, regime_analysis, write_markdown, main
Inputs	saved models, test arrays, tuning summary
Outputs	results/cv_results.csv, final_test_results.csv, band_calibration.csv, regime_analysis.csv, predictions/*.csv, FINAL_RESULTS.md, data/models/model_status.json
Used by	Makefile evaluate; outputs read by prediction, streamlit_app, api
Depends on	build_dataset, registry, ensemble_model, cross_validation, metrics, prediction (band constants)
Pipeline position	Stage 14

File	src/evaluation/plots.py
Purpose	Generate report figures from the result CSVs.
Contains / key items	price history, model comparisons, per-asset actual-vs-predicted, residuals, loss curves, feature importance, strategy curves, over-fitting gap.
Important functions	price_history, comparison(kind), per_asset(asset, status), overfitting_gap, save
Inputs	result CSVs, model_status.json
Outputs	results/figures/*.png (24 files)

Used by	Makefile figures; two figures shown in the dashboard
Depends on	matplotlib, pandas
Pipeline position	Reporting

20.5 src/experiments, src/inference, src/api, src/utlis

File	src/experiments/run_experiments.py
Purpose	Design experiments on validation data only.
Contains / key items	EXP_PARAMS, FEATURE_GROUPS.
Important functions	e1_data_size, e2_feature_ablation, e3_horizon, e4_volatility, write_summary
Inputs	dataset dict per task
Outputs	results/experiments/E1...E4.csv, experiments_summary.md
Used by	Makefile experiments
Depends on	cv_evaluate, build_dataset
Pipeline position	Analysis

File	src/experiments/before_after.py
Purpose	Compare the old 3-year system with the current system on identical unseen days.
Contains / key items	reads archived and current prediction CSVs.
Important functions	metrics_from, main
Inputs	two sets of prediction CSVs
Outputs	results/experiments/before_after.csv
Used by	Makefile experiments
Depends on	evaluate_forecast
Pipeline position	Analysis

File	src/inference/prediction.py
Purpose	Everything that happens when a forecast is requested.
Contains / key items	COMBINED, BASE_MODELS, BAND_SIGMA_FEATURE = "ewma_vol", BAND_RULE, DISCLAIMER.
Important functions	predict_next_day(asset, model), predict_for_date(asset, as_of, model), prediction_history, predict_next_day_safe, available_models, combined_members, live_data_is_current; internal_window, _band, _target_stats, _predict_one, _predict_from_window, _individual, _context
Inputs	features CSV, scaler, models, model_status.json
Outputs	result dict (price, return, direction, band, members, test metrics, disclaimer)
Used by	streamlit_app, api/app, backtesting (constants)
Depends on	config, registry, ensemble_model, market_calendar
Pipeline position	Stage 16-17 (serving)

File	src/api/app.py
Purpose	FastAPI service (title "AI-Powered Commodity Price Predictor API", version 2.1.0).
Contains / key items	pydantic models MemberPrediction, PredictionResponse; CORS open.
Important functions	health_check, list_models, predict_asset, _resolve
Inputs	HTTP requests
Outputs	JSON
Used by	uvicorn (make api)
Depends on	prediction, config
Pipeline position	Serving

File	src/utlis/metrics.py
-------------	----------------------

Purpose	All evaluation metrics.
Contains / key items	return-space and USD-space metrics, tests and the strategy backtest.
Important functions	rmse, mae, mape, r2, directional_accuracy, diebold_mariano, strategy_backtest, evaluate_forecast, qlike, evaluate_vol_forecast, band_calibration, evaluate_task
Inputs	true/predicted returns, prices
Outputs	metric dictionaries
Used by	train_models, cross_validation, backtesting, experiments
Depends on	numpy, scipy
Pipeline position	Stage 10, 14

File	src/utils/inverse_transform.py
Purpose	Reconstruct a price from a return.
Contains / key items	reconstruct_price(log_return, prev_close) = prev_close·exp(log_return)
Important functions	reconstruct_price
Inputs	log return, previous close
Outputs	price
Used by	tests only (prediction.py and backtesting.py apply the same formula inline)
Depends on	numpy
Pipeline position	Utility

File	src/utils/reproducibility.py, logging_config.py
Purpose	Seeds for random/numpy/TensorFlow; centralised logging.
Contains / key items	—
Important functions	set_all_seeds(42), setup_logging, get_logger
Inputs	—
Outputs	—
Used by	registry (seeds); train/tune/ensemble/backtesting/experiments/prediction (logging)
Depends on	tensorflow (lazy import)
Pipeline position	Utility

20.6 app, scripts, tests, notebooks, results

File	app/streamlit_app.py
Purpose	The dashboard (664 lines).
Contains / key items	st.set_page_config, CSS, cached loaders, sidebar, five tabs, footer.
Important functions	load_features, load_csv, load_test_predictions, test_row, layout, pct, usd, usd_md
Inputs	features CSVs, result CSVs, prediction.py
Outputs	web page
Used by	the user (browser)
Depends on	prediction, sync_live_data, market_calendar, config, streamlit, plotly
Pipeline position	Presentation layer

File	scripts/retrain.py
Purpose	Run the entire pipeline as separate subprocesses with logging.
Contains / key items	flags --no-fetch, --no-tune.
Important functions	run(cmd, description), main()
Inputs	command-line flags
Outputs	logs/retrain.log; all pipeline outputs

Used by	developer
Depends on	all pipeline scripts
Pipeline position	Orchestration

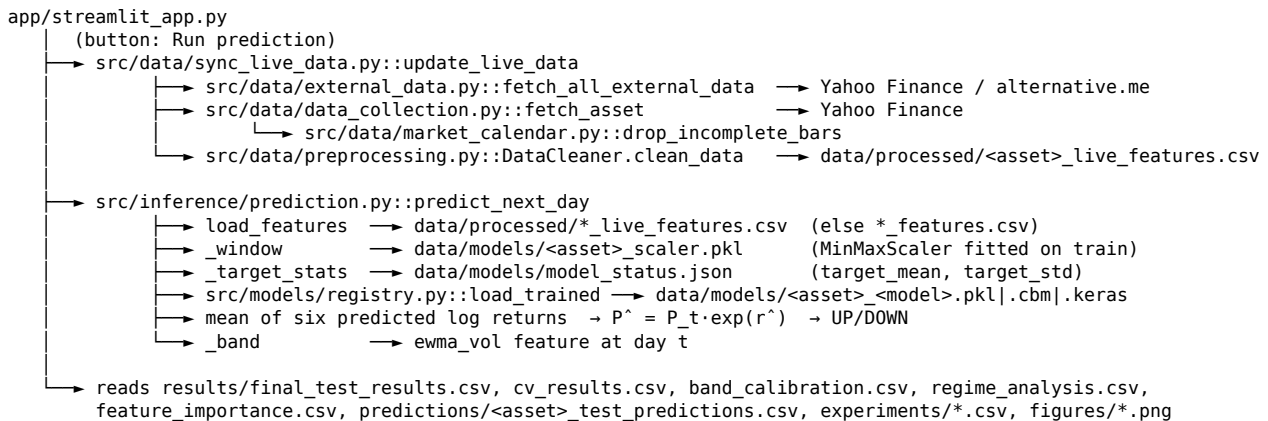
File	tests/ (conftest.py + 5 files)
Purpose	43 automated tests.
Contains / key items	fixtures synthetic_price_data, synthetic_sequences; files test_features.py (8), test_market_calendar.py (3), test_metrics.py (7), test_models_and_inference.py (9), test_sequences_and_split.py (6) — the counts are test functions; some are parameterised, 43 cases total.
Important functions	look-ahead, close-time alignment, complete-bar rule, target alignment, split, scaler fit, reconstruction, metrics, combined inference, band, failure modes
Inputs	synthetic data and the real frozen artefacts
Outputs	pass/fail
Used by	CI, developer
Depends on	the modules under test
Pipeline position	Quality assurance

File	notebooks/01–04
Purpose	Companion notebooks (EDA, feature engineering, training, evaluation) that load and display script outputs.
Contains / key items	9 / 8 / 10 / 11 cells.
Important functions	—
Inputs	result files
Outputs	displayed tables/figures
Used by	readers
Depends on	—
Pipeline position	Reporting (not part of the pipeline)

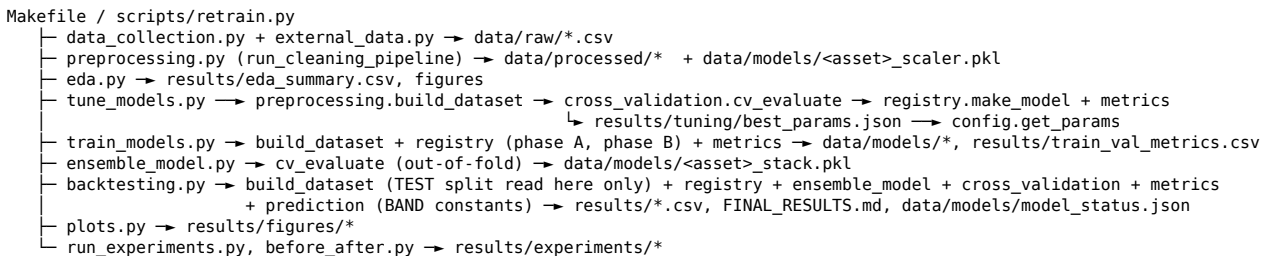
File	results/ and data/models/
Purpose	Outputs. results/FINAL_RESULTS.md is the auto-generated results report; data/models/model_status.json is tracked in git and tells the dashboard which model is best, the target statistics, the band calibration and the Combined test metrics.
Contains / key items	see tree above
Important functions	—
Inputs	—
Outputs	—
Used by	dashboard, API, docs
Depends on	backtesting.py writes them
Pipeline position	Artefacts

21. File-to-File Connection Map

21.1 Serving path (what runs when you use the dashboard)



21.2 Training / evaluation path



21.3 Who imports whom (verified from the import statements)

Module	Imports (project modules)	Imported by
config	—	all modules
market_calendar	—	data_collection, external_data, prediction, streamlit_app, tests
data_collection	config, market_calendar	sync_live_data
external_data	config, market_calendar	sync_live_data
preprocessing	config	tune_models, train_models, ensemble_model, backtesting, run_experiments, sync_live_data, tests
sync_live_data	config, preprocessing, data_collection, external_data	streamlit_app
registry	config, reproducibility	train_models, cross_validation, ensemble_model, backtesting, prediction, run_experiments
metrics	—	train_models, cross_validation, backtesting, run_experiments, before_after
cross_validation	config, registry, metrics	tune_models, ensemble_model, backtesting, run_experiments
ensemble_model	config, preprocessing, cross_validation, registry, logging_config	backtesting, prediction
prediction	config, registry, ensemble_model, market_calendar, logging_config	streamlit_app, api/app, backtesting (BAND_SIGMA_FEATURE, BAND_RULE)
backtesting	config, preprocessing, registry, ensemble_model, cross_validation, metrics, prediction	Makefile, retrain.py

Note the one “reverse” dependency: backtesting.py imports the band constants from prediction.py, so the evaluation scores exactly the band that the dashboard serves.

22. Important Functions

Function	File	Purpose	Input	Output
fetch_asset	data_collection.py	download OHLCV, drop incomplete bar, save CSV	asset, start, out_dir	DataFrame / CSV
fetch_all_external_data	external_data.py	download macro + Fear & Greed	out_dir	dict of DataFrames / CSVs
drop_incomplete_bars	market_calendar.py	remove the still-running day	DataFrame, asset type	filtered DataFrame
DataCleaner.clean_data	preprocessing.py	clean + build every feature	raw DataFrame	feature DataFrame
DataCleaner._align_external	preprocessing.py	close-time-safe macro alignment	external series	aligned series
DataCleaner.add_rolling_volatility	preprocessing.py	rolling, HAR and EWMA volatilities	—	columns volatility_*, rv_*, ewma_vol
DataCleaner.normalize_and_split	preprocessing.py	chronological split + train-only MinMax scaling	features	scaled train/val/test, scaler
_task_target	preprocessing.py	create the forward-looking target	log_return series, task	target series (t → t+h)
build_dataset	preprocessing.py	windows, target, standardisation, split by target dates	asset, task	dict of arrays
get_params	config.py	tuned params or defaults	model, asset, task	dict
cv_evaluate	cross_validation.py	walk-forward evaluation	model, params, dataset	fold metrics (+OOF)
tune	tune_models.py	grid search for one (asset, model)	asset, model, data	best params, score
train_asset	train_models.py	phase A + phase B training and saving	asset, models	metric rows, importances
make_model, load_trained	registry.py	create / load a model object	name, params	model object
LightGBMModel.fit / CatBoostModel.fit / _KerasSequenceModel.fit	registry.py	early-stopped two-phase fitting	train/val arrays, final	fitted model
evaluate_forecast	metrics.py	full metric set for a forecast	true/pred returns, prices	dict of metrics
diebold_mariano	metrics.py	significance of RMSE difference	true, pred A, pred B	(statistic, p)
test_predictions	backtesting.py	predict test set with all models + Combined	asset, data	preds, fit infos
main	backtesting.py	full final evaluation, writes results and model_status.json	—	CSV/JSON/MD files
fit_stack	ensemble_model.py	fit the meta-model	asset, bases	weights, stack.pkl
update_live_data	sync_live_data.py	refresh live features	asset	bool
predict_next_day	prediction.py	forecast for the next trading day	asset, model	result dict
predict_for_date	prediction.py	demo on a historical test day with reveal	asset, date	result dict + actual outcome
_window, _band, _predict_from_window	prediction.py	scaled input window; $\pm 1\sigma$ band; average of members	features / models	arrays, band, $\hat{r}$
predict_asset	api/app.py	REST endpoint	asset, model	PredictionResponse
dashboard tabs (with tab_fc: ... tab_meth:)	streamlit_app.py	render the five tabs	prediction dict, result CSVs	web UI

23. Libraries and Technologies

Listed in `requirements.txt` (versions as pinned there). “Used in” was checked by reading the imports.

Technology	Version	Why we use it / where
Python	3.9.6 in the local venv; CI uses 3.11; README badge 3.9+	Language of the whole project
pandas	2.3.3	time-series tables, rolling/EWM features, CSV I/O (everywhere)
NumPy	1.26.4	arrays, windows, math
scikit-learn	1.6.1	MinMaxScaler, Ridge, RandomForestRegressor, TimeSeriesSplit
joblib	1.5.3	saving scalers and sklearn/LightGBM models (.pkl)
TensorFlow / Keras	2.16.2	GRU and LSTM networks (.keras files)
LightGBM	4.6.0	gradient-boosted trees
CatBoost	≥ 1.2.0	gradient-boosted trees (.cbm files)
SciPy	≥ 1.10	binomial test, t-distribution (DM test), skew/kurtosis
statsmodels	0.14.6	ARIMA, ADF test, ACF/PACF
yfinance	1.2.0	Yahoo Finance price download
ta	≥ 0.11.0	Bollinger Bands, ATR, ADX, ROC indicators
Streamlit	1.50.0	dashboard
Plotly	6.7.0	interactive charts in the dashboard
Matplotlib	3.9.4	static report figures (plots.py, eda.py)
FastAPI, Uvicorn (+ pydantic)	≥ 0.100 / ≥ 0.20	REST API
tzdata	≥ 2024.1	time-zone database for zoneinfo (America/New_York) in market_calendar.py
pytest	≥ 7.0	test suite
Jupyter	1.1.1	companion notebooks
requests	2.32.5	listed in requirements; no direct import in the project code (the Fear & Greed call uses <code>urllib</code>)

24. Model Files / Saved Artefacts

Artefact	Location / format	Created by	Loaded by
Ridge, Random Forest, LightGBM models	data/models/<prefix>_{ridge,randomforest,lightgbm}.pkl — joblib dict {model, params, fit_info}	BaseModel.save in train_models.py (phase B)	BaseModel.load ← load_trained
CatBoost	<prefix>_catboost.cbm + .cbm.json (params, fit_info)	CatBoostModel.save	CatBoostModel.load
GRU / LSTM	<prefix>_{gru,lstm}.keras + .keras.json (params, fit_info, loss history)	_KerasSequenceModel.save	keras.models.load_model + JSON
Scalers	data/models/<prefix>_scaler.pkl (MinMaxScaler fitted on training rows)	DataCleaner.save_data	prediction._window, build_dataset
Stacked meta-model	<prefix>_stack.pkl {meta, bases, weights}	fit_stack	StackedModel
Model status / config	data/models/model_status.json — per asset: best single model, selection rule, CV and test metrics, Combined metrics, feature list, target mean/std, band calibration	backtesting.py	config.load_model_status, prediction, dashboard, API
Tuned hyper-parameters	results/tuning/best_params.json	tune_models.py	config.get_params
Feature lists	stored inside model_status.json["<asset>"]["features"]; at prediction time the columns are re-derived (all non-LEVEL columns) and checked against the scaler	backtesting.py	dashboard Methodology tab; _window check
Encoders	None — there are no categorical features	—	—

Totals present on disk: 18 trained models (6 per asset), 3 scalers, 3 stack files. **If an artefact is missing:** a missing scaler/model raises `FileNotFoundError` from joblib/Keras (API → HTTP 503, dashboard → “Prediction failed: ...”); a missing `model_status.json` target statistic raises an explicit error; missing models are skipped when assembling the Combined member list (see Chapter 16 caveat). Fix: make preprocess train evaluate.

25. Configuration and Environment

- **Environment variables:** none required. `.env.example` documents one optional variable, `TF_CPP_MIN_LOG_LEVEL` (the code also sets it to 3 via `os.environ` in several scripts). **There are no API keys and no secrets** — Yahoo Finance and alternative.me are free and unauthenticated.
- **Configuration files:** `config.py` (all settings), `results/tuning/best_params.json` (tuned parameters), `data/models/model_status.json` (served-model status), `.claude/launch.json` (preview server entries), `.vscode/settings.json` (`makefile.configureOnOpen: false`).
- **Python version:** local virtual environment: Python 3.9.6; CI: 3.11; README states 3.9+.
- **Dependency management:** `requirements.txt` (direct, pinned) and `requirements-lock.txt` (full freeze, 164 lines).
- **Install:** `python -m venv venv && source venv/bin/activate && pip install -r requirements.txt` (make install does the pip step).
- **CI:** `.github/workflows/ci.yml` runs install, import checks and pytest.

26. How to Run the Project

All commands below are taken from README.md and the Makefile.

Step 1 — Install requirements

```
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
```

Step 2 — Configure environment

Nothing is required. Optionally copy .env.example to .env (git-ignored) to set TF_CPP_MIN_LOG_LEVEL.

Step 3 — Prepare / download the dataset

```
make collect-data # downloads OHLCV + macro + Fear&Greed into data/raw (needs internet; Yahoo may rate-limit)
make preprocess  # features + frozen chronological split + scaler (from data/raw)
make eda         # optional: ADF tests, ACF/PACF, distributions
```

The repository already contains the frozen data/raw/ and data/processed/ files used for all reported results; running collect-data again downloads newer data and would change the results (Yahoo also revises history).

Step 4 — Train models (if required)

```
make tune # optional, ~30 min: walk-forward hyper-parameter search (reuse results/tuning/best_params.json to skip)
make train # two-phase training of all six models
make stack # stacked-ensemble experiment
make evaluate # ONE evaluation on the test set → results/*, data/models/model_status.json
make figures # report figures
make test # 43 tests
# or everything: make pipeline / python scripts/retrain.py [--no-tune] [--no-fetch]
```

On a fresh clone (data/ is git-ignored): make collect-data preprocess train stack evaluate (≈ 15 min without tuning, using the committed best_params.json).

Step 5 — Start Streamlit

```
make serve # = streamlit run app/streamlit_app.py
```

Step 6 — Open the application

Open **http://localhost:8501**, choose an asset in the sidebar, press **Run prediction**. Optional: make api → **http://localhost:8000/docs**; make predict prints the combined forecast of all assets in the terminal.

Tip for the viva: the dashboard works offline from the stored data. “Run prediction” tries to download the newest bar; if Yahoo is unreachable it falls back to the stored live/frozen features and says so. “Predict a Day” always works offline.

27. Complete End-to-End Example (real output)

The numbers below are the **actual output** of the project's own code, produced for this guide on 2026-10-03 by calling `predict_next_day("Bitcoin")` on the stored live-feature file (which ends on the 2026-09-20 bar — the last time live data was synced). They are an illustration of the mechanism, not a forecast about today.

Scenario A — the user selects Bitcoin and presses Run prediction

1. **Data retrieved:** data/processed/btc_live_features.csv (last complete bar 2026-09-20, close \$81,142.61; source label "live download"). In a fresh session the dashboard would first refresh from Yahoo if the file were not current.
2. **Preprocessing:** the file already contains the cleaned data and all indicators (computed by `DataCleaner.clean_data`).
3. **Features generated:** 29 model features for each day, e.g. today's return, RSI, ADX, EMA ratios, Bollinger %B, volatilities, S&P 500/VIX returns, Fear & Greed, day-of-week sin/cos, volume change.
4. **The latest feature row, conceptually:** one row of 29 numbers describing the market on 2026-09-20, scaled to roughly 0–1 with the training-time scaler; together with the previous 29 days it forms a 30×29 window (for GRU/LSTM); Ridge/RF/LightGBM/CatBoost see only the last row.
5. **Which models receive it:** all six.
6. **What each model produced** (predicted next-day return): Ridge +0.29 %, Random Forest +0.13 %, LightGBM +0.17 %, CatBoost +0.24 %, GRU +0.08 %, LSTM +0.02 %.
7. **Combination:** mean = **+0.156 %** (0.00156 as a log return).
8. **Final price:** $81,142.61 \times \exp(0.001557) = \mathbf{\$81,269.08}$.
9. **UP/DOWN:** +0.156 % > 0 → **UP**.
10. **Band:** σ_t = EWMA volatility = 2.21 % → $\pm 1\sigma$ range **\$79,491 - \$83,086** (the "market noise" scale is ~14× the predicted move).
11. **In Streamlit:** a card "Final prediction — combination of 6 models · close on 2026-09-20 \$81,142.61 → predicted close for 2026-09-21: \$81,269.08 UP ▲ +0.16 %", the band caption with its test coverage (73.9 %), the per-model table, the 60-day chart, and the reliability table vs the random walk ("Beats random walk? No (DM p = 0.49)").

Scenario B — "Predict a Day" on an unseen test day (real output)

For Gold standing on 2026-05-12 (inside the unseen test period; the frozen dataset is used and only data up to that day): close \$4,677.60 → predicted \$4,681.18 (+0.076 %, UP); the actual close on 2026-05-13 was \$4,697.70 (+0.43 %), prediction error −0.35 %, direction HIT, actual inside the $\pm 1\sigma$ band. One lucky hit proves nothing: over all 143 Gold test days the direction hit-rate is 50.3 %.

Scenario C — what the same Combined forecast looks like for the other assets (stored data)

Asset	As-of bar	Close	Combined forecast	Direction	$\pm 1\sigma$ band	σ_t
Bitcoin	2026-09-20	\$81,142.61	\$81,269.08 (+0.16 %)	UP	\$79,491 – \$83,086	2.21 %
Gold	2026-09-18	\$4,424.90	\$4,431.14 (+0.14 %)	UP	\$4,374 – \$4,489	1.31 %
Silver	2026-09-18	\$66.56	\$66.61 (+0.08 %)	UP	\$65.11 – \$68.14	2.28 %

28. Strengths of the Project

Each strength below is backed by something that exists in the code or results.

Strength	Evidence
Leakage-aware design , the hardest part of financial ML	close-time rule for macro data and High/Low features (<code>config.py</code> , <code>preprocessing.py</code>); train-only scaler; target never in the window; tests; independent truncation test; the earlier 62 % Silver result was traced to a leak and removed.
Time-series-correct validation	frozen chronological split by date; 4-fold expanding-window walk-forward with <code>TimeSeriesSplit</code> ; purge/embargo for multi-day targets; test set read by one script only.
Honest baseline comparison	random walk, historical mean and ARIMA in every table; Diebold-Mariano test; direction hit-rate with binomial p-value; “UP calls” share to expose drift-only forecasts.
Multiple model families under identical conditions	six models + three baselines + a stacking experiment through one registry interface, same samples, same metrics.
Principled hyper-parameter search	grid search with walk-forward CV on train+val; per-config logs; early stopping in a two-phase fit so validation does not leak into its own monitor.
Served forecast needs no model choice	equal-weight Combined forecast; nothing selected on the test set.
Useful uncertainty band	EWMA-volatility band with measured coverage 74–76 % on the test set versus 51–89 % for a fixed-width band (nominal 68 %).
Separation of training and inference	artefacts saved to <code>data/models/</code> ; <code>prediction.py</code> loads them; the dashboard never trains.
Reproducibility and engineering quality	fixed seeds, frozen raw data, lock file, Makefile, <code>retrain.py</code> , CI workflow, 43 tests, bit-for-bit reproduction verified by the project’s own audit (CHANGELOG).
Defensive inference code	explicit errors for missing stats, feature mismatch, non-finite output, corrupt live data; atomic file writes; live download throttling.
Transparent dashboard	every forecast is shown beside its held-out metrics, the random-walk comparison and a disclaimer; the “Honest reading” text is generated from the numbers.

29. Limitations / Weaknesses

Area	Limitation (supported by the project)
Predictability	Next-day returns are close to unpredictable: $R^2 \approx 0$, no model beats the random walk, direction $\approx$ coin flip.
Dataset	Only daily public data ($\approx 2,000$ – $3,000$ rows per asset). No news, order-book, on-chain, or fundamental data. Yahoo is a free vendor: closes are revised (up to 0.75 % on one Bitcoin day measured by the project) and results depend on the frozen download.
Futures series	GC=F / SI=F are continuous front-month series; contract-roll jumps are not adjusted. The dashboard warns that live closes can differ after a roll.
Features	Hand-engineered technical/volatility/macro features; the ablation experiment shows none matters at a 1-day horizon. Futures volume is unused. Weekend S&P/VIX values for Bitcoin repeat Friday's return.
Models	Models stop after 1–10 epochs/trees and predict almost constant values (std ≈ 0.1 % vs 1.7–3.2 %). Grids are small (GRU/LSTM: 2 configs each); Ridge α and RF depth sit at the edge of their grids. LightGBM subsample has no effect without subsample_freq.
Validation	Short windows (109–160 validation days, 143–207 test days): a 3–4 point difference in hit-rate is within noise. All assets share one calendar split, one market regime. 30 model $\times$ asset direction tests were looked at; one (LightGBM, Silver, $p = 0.02$) is “significant” — expected by chance.
Test-set history	Earlier project versions were evaluated on the same window (3-year data; leaky macro version). The final rules use validation only, but the test window is not completely virgin in the project's history.
Over-fitting	Controlled via early stopping and strong regularisation; the large train $\rightarrow$ validation RMSE jump for the metals reflects higher validation-period volatility (the random walk jumps too), not over-fitting in the classical sense.
Market volatility / external events	Sudden news, geopolitical shocks and regime changes are not modelled; the EWMA band adapts only after volatility rises.
Frequency	Daily only; no intraday forecasts; holidays are not modelled in the “next trading day” label (BDay).
Computation	Tuning takes ≈ 30 min (GRU/LSTM dominate); TensorFlow is a heavy dependency; models are re-loaded from disk on each prediction call (no in-memory cache).
Code details	Combined silently averages whatever model files exist; create_sequences and reconstruct_price are exercised only by tests (the pipeline uses equivalent inline code); requests is listed in requirements but not imported.

30. What Our System Does NOT Do

- It does **not** guarantee future prices, profit or direction. Test-set direction accuracy is 46.9 / 50.3 / 53.8 %, none significant.
- It does **not** beat the “tomorrow = today” forecast (all Diebold–Mariano $p \geq 0.17$ across every model and asset).
- It is **not** a professional trading system: no position sizing, risk management, execution model; the long/flat backtest of the served forecast lost money on all three assets (−5.1 %, −7.9 %, −6.0 %) versus buy-and-hold +14.5 %, −10.6 %, −12.1 %.
- It does **not** predict unexpected events (news, geopolitics, policy) — no such variables are inputs.
- It does **not** forecast beyond one trading day in production (5-day and volatility targets are experiments only).
- It does **not** continuously learn or log live forward predictions (an automatic forward-prediction log was built and then removed by the team).
- It does **not** use intraday data, order books, on-chain data, fundamentals or news sentiment.
- Historical relationships can change; a pattern found in 2018–2025 may not hold in 2026 or later.

Safe sentence for the viva: “We built and validated a pipeline to test whether next-day returns are predictable. We found they are not, in a statistically measurable way, using public daily data. The deliverable is the pipeline, the evidence, and a calibrated uncertainty band.”

31. Common Viva Questions (50+ questions with answers)

Format: **Short** = the sentence to say first; **Detail** = what to add if the examiner continues. Numbers are the project's actual results.

31.1 Basic questions

Q1. In one minute, what does your project do?

Short: It downloads daily Bitcoin, Gold and Silver prices, builds leak-free features, trains six ML models to predict the next-day return, combines them, and shows the forecast with its measured reliability in a Streamlit dashboard.

Detail: We compare every model against the random walk ("tomorrow = today") on unseen days. The honest finding is that none beats it; the dashboard therefore shows each forecast together with the random-walk comparison and an uncertainty band.

Q2. What problem are you solving?

Short: Whether machine learning can predict next-day prices of Bitcoin, Gold and Silver better than a naive forecast — answered with a rigorous, leak-free evaluation.

Detail: Many projects claim high accuracy because of data leakage or because they predict price levels. We predict returns, validate chronologically and test statistically.

Q3. What exactly are you predicting?

Short: The next trading day's log return, $\ln(P_{t+1}/P_t)$, which we convert to a price with $P_t \cdot \exp(\text{prediction})$.

Detail: Defined in `config.TARGET_COL`, created in `preprocessing._task_target` (shift by -1), reconstructed in `prediction.py`.

Q4. Why these three assets?

Short: They are three widely traded assets with free daily history, and technically they cover one 24/7 crypto market and two exchange-traded futures with a different daily close time, which makes the leakage-safe design (close-time rule) non-trivial.

Detail: The code treats them as type: `crypto` (Bitcoin) and type: `commodity` (Gold, Silver) in `ASSET_CONFIG`; alignment of macro data differs between the two types.

Q5. Can I make money with your system?

Short: No. It is an academic study; it does not beat the random walk and the disclaimer says it is not financial advice.

Detail: Our long/flat backtest with 10 bps costs lost money on all three assets for the served forecast, and direction accuracy is not significantly above 50 %.

Q6. What is your main result?

Short: No model — single or combined — significantly beats the random-walk baseline at a one-day horizon.

Detail: Combined RMSE (return) is within ± 0.4 % of the baseline, DM $p = 0.49 / 0.97 / 0.46$, $R^2 \approx 0$, direction 46.9 / 50.3 / 53.8 % ($p = 0.83 / 0.50 / 0.20$).

Q7. If the models do not beat the baseline, is the project worthless?

Short: No — a correct negative result is a valid scientific result, and the project delivers the audited pipeline plus a useful uncertainty band.

Detail: Financial theory (weak-form efficiency) predicts exactly this. The value is the method: leakage audit, walk-forward validation, significance tests, and a calibrated $\pm 1\sigma$ band (74–76 % test coverage vs 68 % nominal).

31.2 Dataset questions

Q8. Where does your data come from?

Short: Yahoo Finance via the yfinance library for prices and macro series; the alternative.me API for the Fear & Greed index.

Detail: Tickers BTC-USD, GC=F, SI=F, DX-Y.NYB, CL=F, ^VIX, ^TNX, ^GSPC. Free, no API keys. Downloaded by data_collection.py and external_data.py into data/raw/.

Q9. How many rows and which dates?

Short: Daily data from 2018-01-01: 3,177 Bitcoin rows to 2026-09-12 and 2,186 rows each for Gold and Silver to 2026-09-11.

Detail: After feature warm-up: 3,147 / 2,156 / 2,156 rows. Splits: Bitcoin 2,750 / 160 / 207 samples; metals 1,874 / 109 / 143.

Q10. What are the columns?

Short: timestamp, open, high, low, price (close) and volume.

Detail: Macro files have timestamp and price (Fear & Greed: timestamp and fear_greed). Price levels are kept for charts but are not model inputs; the model uses 28-29 derived stationary features.

Q11. How do you handle missing values, duplicates and outliers?

Short: Duplicates are dropped by timestamp; missing macro returns are filled with 0 and missing Fear & Greed with 50; warm-up rows are dropped; outliers are deliberately kept.

Detail: The raw files had 0 nulls and 0 duplicates. A validation function raises an error on NaN/inf/too-few rows. Only volume change is clipped (± 3). Fat tails are real market behaviour, so we do not remove them.

Q12. What is the “complete-bar rule”?

Short: Yahoo returns today’s still-running row as if finished; we drop it so the data always ends on a finished bar.

Detail: market_calendar.complete_bar_cutoff: crypto uses the UTC day; futures/US series are final after 17:15 ET. Without it we would train/evaluate on intraday snapshots.

Q13. Why futures (GC=F, SI=F) instead of spot gold and silver?

Short: We use Yahoo Finance’s COMEX futures tickers (GC=F, SI=F): free, long daily history through the same library as the other data. The repository does not document any other reason.

Detail: Consequence: contract roll jumps are in the data (not adjusted) — listed as a limitation; the close is the 13:30 ET COMEX settlement, which drives the close-time rule.

Q14. Why is the data daily and not intraday?

Short: Daily data is freely available and long enough (8+ years); the project’s horizon is one trading day.

Detail: Intraday free data from Yahoo is limited in history; intraday forecasting would be a different project (future work).

31.3 Machine learning questions

Q15. Why machine learning, and why regression rather than classification?

Short: We want to test whether ML can extract patterns from market features; regression of the return gives both size and direction and lets us reconstruct a price.

Detail: A classifier of UP/DOWN would throw away magnitude and could not produce a price or compare against RMSE. Direction is still derived (sign of the predicted return) and tested with a binomial test.

Q16. Why predict returns instead of prices?

Short: Prices are non-stationary ($ADF\ p \approx 1$) while returns are stationary; predicting prices makes any model look good ($R^2 \approx 0.95$ for “tomorrow = today”).

Detail: Test values of price-like features fall outside the training range; ratios and returns do not. The price forecast is recovered exactly with $\exp()$.

Q17. What is overfitting and how did you control it?

Short: Overfitting is learning noise in the training data so that new data is predicted badly; we used strong regularisation, shallow trees, dropout, early stopping and out-of-time validation.

Detail: Ridge $\alpha = 100$, RF depth 3 and ≥ 30 samples per leaf, LightGBM/CatBoost early stopping, dropout 0.2–0.4, patience-6 early stopping for GRU/LSTM, and walk-forward tuning on data later than the training window.

Q18. Why do your models stop after 1 epoch or 10 trees?

Short: Because validation error did not improve — there is almost no learnable signal, so early stopping keeps the model near “predict the average”.

Detail: Final stopping points: LightGBM 81 / 10 / 10 trees, CatBoost 50 / 606 / 82, GRU 1 / 8 / 9 epochs, LSTM 2 / 1 / 4 epochs. It matches the tiny spread of predictions (std ≈ 0.1 % vs 1.7–3.2 % actual).

Q19. Why six different models?

Short: To compare linear, tree-based, boosting and recurrent families fairly, and to average their errors.

Detail: The question “does model complexity help?” is part of the research. Result: it does not — all six ML models lie within about 0.5 % of the random walk in walk-forward RMSE.

Q20. Why average the models with equal weights instead of picking the best?

Short: The “best” differs by asset and by split and the differences are not significant, so selecting one would be selecting noise; equal weights need no fitting.

Detail: Nothing is chosen on the test set. The best single model by walk-forward validation (CatBoost, CatBoost, GRU) is reported separately for comparison.

Q21. Why not use a bigger deep-learning model?

Short: With $\sim 1,900$ – $2,750$ samples a deeper network over-fits; an earlier 2×100 -unit version over-fitted badly (documented in `model_gru.py`).

Detail: The GRU/LSTM here have 6.5–8.5 thousand parameters. Even these stop after 1–9 epochs.

31.4 Feature engineering questions

Q22. Which features do you use?

Short: 28–29 stationary features: returns and lags, 20-day momentum, rolling/HAR/EWMA volatility, RSI, ADX, ROC, MACD ratios, EMA ratios, Bollinger %B/width, ATR/price, range, macro returns and sentiment.

Detail: Exact lists per asset are in Chapter 7. Raw prices, volume levels and indicator levels are excluded (LEVEL_COLUMNS).

Q23. What is the close-time rule and why does it matter?

Short: A feature may only use information known when the asset’s daily close is fixed; for gold and silver (13:30 ET settlement) we therefore use the previous day’s macro values.

Detail: Bitcoin closes at 00:00 UTC after all US markets, so same-day macro values are known. Ignoring the rule produced a fake 62 % direction accuracy for Silver; fixing it removed that.

Q24. Which scaler do you use and where is it fitted?

Short: MinMaxScaler (0–1), fitted on the training rows only and applied to validation, test and live data.

Detail: `normalize_and_split` fits; `<prefix>_scaler.pkl` stores it; `prediction._window` reloads it. The test data goes outside [0,1] in 0.02–3.4 % of cells — proof it was not fitted on test.

Q25. Why are price levels not used as inputs?

Short: They are non-stationary: future prices fall outside the training range, which breaks tree and linear models.

Detail: We use ratios (price/EMA – 1, MACD/price, ATR/price) that stay in a comparable range over time.

Q26. Which features matter most?

Short: No stable winners: each model ranks differently (e.g. Gold: `return_20d`, `tnx_return`; Silver: `ema14_ratio`, `macd_hist_norm`).

Detail: Feature ablation (E2) shows removing any feature group changes validation RMSE by only -0.4 % ... $+0.05$ %, consistent with “no signal”.

31.5 Model questions

Q27. How does a Random Forest work?

Short: Many shallow decision trees, each trained on a random sample of rows and features; the prediction is their average.

Detail: Bootstrapping decorrelates trees; averaging reduces variance. Ours: 300 trees, depth 3, ≥ 30 samples per leaf.

Q28. How do LightGBM and CatBoost work?

Short: Gradient boosting: trees are added one after another, each fitted to the errors of the current ensemble, scaled by a small learning rate.

Detail: Early stopping on validation decides the number of trees. LightGBM grows leaf-wise; CatBoost uses symmetric trees and is less prone to over-fitting.

Q29. How do GRU and LSTM work and what is the difference?

Short: Both read the 30-day window in order and keep a hidden memory controlled by gates; GRU has 2 gates and no cell state, LSTM has 3 gates plus a cell state.

Detail: Both are followed by Dropout $\rightarrow$ Dense(16, ReLU) $\rightarrow$ Dense(1). GRU is lighter (6.5k vs 8.5k parameters). Neither beats the baseline here.

Q30. What is Ridge regression and why is alpha 100?

Short: Linear regression with an L2 penalty that shrinks weights; $\alpha = 100$ was the best (and the largest) value in the grid.

Detail: It means the data prefers maximum shrinkage — predictions close to the mean — a further hint of low signal.

Q31. What is ARIMA and what did you do with it?

Short: A classical statistical model using past values and past errors; we use ARIMA(p,0,q) on returns with the order chosen by AIC as a baseline.

Detail: Orders on the final run: Bitcoin (2,0,0), Gold (1,0,1), Silver (2,0,3); forecasts are walk-forward one step ahead. It has the highest test RMSE for Bitcoin and Silver.

Q32. What is the stacked ensemble and why is it not the served model?

Short: A non-negative Ridge meta-model trained on out-of-fold predictions of four base models; it is an experiment, not served.

Detail: It is itself a blend of some members, its learned weights change wildly across assets (e.g. Gold CatBoost 1.18, GRU 0.59), and it is the best on Bitcoin but the worst on Gold — noise.

31.6 Time-series questions

Q33. Why do you split chronologically and not randomly?

Short: Random splitting would place neighbouring days in train and test and leak market state; chronological order mimics real use.

Detail: Split dates are frozen in config.py: train $\leq$ 2025-09-10, validation $\leq$ 2026-02-17, test afterwards. No shuffling anywhere.

Q34. What is walk-forward (expanding-window) validation?

Short: We repeatedly train on all data before a block and test on that block, moving forward in time; we use 4 folds.

Detail: Implemented with TimeSeriesSplit(4) in cross_validation.py; used for hyper-parameter tuning and the model comparison; only train+val is involved.

Q35. What is stationarity and what did you find?

Short: A series is stationary if its statistics do not drift; prices are not (ADF $p \approx 0.97$ –1.0) but log returns are ($p \approx 0$).

Detail: This is why the target is the return. Source: eda.py $\rightarrow$ results/eda_summary.csv.

Q36. What is data leakage and where could it have happened here?

Short: Using information from the future (or from the test set) when building inputs; we guard against it in features, scaling, splitting and tuning.

Detail: The one real leak was same-day macro data and full-session High/Low for the futures, whose close is at 13:30 ET; it was fixed and documented (results/archive_sameday_macro_leak/).

Q37. How do you know there is no leakage now?

Short: Unit tests plus a truncation test: rebuilding features from data cut at a date reproduces the full-data features to 1e-11.

Detail: test_no_lookahead_in_any_feature, test_scaler_fitted_on_train_only, test_create_sequences_target_is_next_row_and_never_in_window, close-time tests; the test suite passes (43). Residual risk: Silver's same-day gold_return assumes a 5-minute settlement gap is negligible.

31.7 Evaluation questions

Q38. Which metrics do you use and why?

Short: RMSE/MAE/R² in return space (what we predict), MAE/RMSE/MAPE in USD (what users see), directional accuracy with a binomial test, Diebold-Mariano test against the random walk, a strategy backtest and band coverage.

Detail: Return-space RMSE is the main comparison metric; USD metrics are reported but misleading because the naive forecast scores the same.

Q39. What is R² and why is it about zero?

Short: R² compares the model with predicting the mean; ≈ 0 means the model explains none of tomorrow's variation.

Detail: Combined: +0.006 (Bitcoin), -0.001 (Gold), +0.005 (Silver) on the test set; walk-forward values are -0.004 ... -0.001.

Q40. Silver's direction accuracy is 53.8 % — is that good?

Short: No: with 143 days the p-value is 0.20; 53.8 % is within chance.

Detail: The standard error of a hit-rate near 50 % on 143 days is about 4 points. One model (LightGBM, Silver) reaches $p = 0.02$, but among ~ 30 tests one such result is expected by chance and it was not the validation-selected model.

Q41. What is the Diebold-Mariano test?

Short: A test of whether two forecasts have the same squared-error accuracy; we compare each model with the random walk.

Detail: Statistic = $\text{mean}(d)/\sqrt{(\text{var}(d)/n)}$ with $d = e^2_{\text{model}} - e^2_{\text{naive}}$ and a small-sample correction (metrics.diebold_mariano). Our p-values are 0.17 ... 0.99 — never significant.

Q42. Your MAPE is only about 1.5 % — isn't that excellent?

Short: No — "tomorrow = today" has the same MAPE (1.52 % for Bitcoin), so it shows no skill.

Detail: Bitcoin test MAE: \$1,063.94 (Combined) vs \$1,066.42 (random walk). Daily moves are $\sim 2\%$, so any forecast near today's price scores $\sim 1.5\%$.

Q43. Why does the predicted line follow the actual line so closely in the chart?

Short: Because each prediction starts from today's actual close; in price space it always looks accurate.

Detail: The dashboard says this on the chart. Skill must be judged in return space, where predictions barely move (std 0.1 % vs 2 %).

Q44. Why does Gold say UP on 97 % of days?

Short: The averaged prediction is a small positive number almost every day (the average drift), so the direction call is nearly constant.

Detail: Gold had 50 % up days in the test window, so a constant UP scores $\sim 50\%$ by construction. We added the "UP calls" column to expose this.

Q45. What does the strategy backtest show?

Short: Going long when the forecast is positive (10 bps cost) lost money for the served forecast on all three assets: -5.1 %, -7.9 %, -6.0 %.

Detail: Buy-and-hold was +14.5 %, -10.6 %, -12.1 %. It is a sanity check, not a trading claim; some single models are positive by chance (e.g. LightGBM Silver +10.1 %).

Q46. Is your $\pm 1\sigma$ uncertainty band any good?

Short: Yes, comparatively: it contained the actual close on 73.9 % / 75.5 % / 75.5 % of test days (nominal 68.3 %), better than a fixed-width band (88.9 / 51.0 / 54.5 %).

Detail: σ_t is the EWMA ($\lambda = 0.94$) volatility known on day t ; the band is $P_t \cdot \exp(\hat{r} \pm \sigma_t)$. It is slightly conservative on the test set; on validation Silver covered only 57.8 %. Volatility is the part of the problem that is predictable.

31.8 Architecture / code questions

Q47. Explain the architecture of your code.

Short: data layer → preprocessing/feature engineering → model registry → training/tuning → single test evaluation → inference module → Streamlit/API.

Detail: config.py is the single source of settings; preprocessing.build_dataset is the single loader; backtesting.py is the single test-set reader; prediction.py is the single serving module (Chapter 21).

Q48. What happens when I click “Run prediction”?

Short: The app refreshes the latest data if needed, loads the scaler and six models, each predicts the next-day return, the six are averaged, the price and UP/DOWN are computed and shown with a band and reliability table.

Detail: Functions: update_live_data → predict_next_day → _window → _predict_one (×6) → mean → $P_t \cdot \exp(\hat{r})$ → _band (Chapter 17).

Q49. What if Yahoo Finance is down?

Short: The download fails gracefully, the existing data file is kept, and the dashboard predicts from the stored data and says so.

Detail: fetch_asset returns an empty frame without overwriting; update_live_data returns False; the sidebar shows a warning. test_live_sync_survives_network_failure.

Q50. How are models saved and loaded?

Short: sklearn/LightGBM with joblib (.pkl), CatBoost as .cbm, Keras as .keras, each with a JSON side-car; the scaler as .pkl.

Detail: All in data/models/, named <prefix>_<model>; registry.load_trained loads; model_status.json stores the target mean/std needed to un-standardise predictions.

Q51. How is UP/DOWN decided?

Short: UP if the combined predicted log return is positive, otherwise DOWN.

Detail: Equivalent to predicted price > current price. There is no threshold; it is not a trading signal.

Q52. How do you ensure reproducibility?

Short: Fixed seeds, frozen raw data, a single split definition, a lock file, a Makefile pipeline and tests.

Detail: set_all_seeds(42), random_state=42, random_seed=42; the project’s audit reproduced features and test results bit-for-bit (CHANGELOG, “Final audit”).

Q53. What does your API do?

Short: A FastAPI service with /health, /models and /predict/{asset} returning the same forecast as the dashboard in JSON.

Detail: Errors: 404 for unknown asset or model, 503 for missing artefacts. make api serves it at port 8000.

31.9 Limitations questions

Q54. What are the biggest weaknesses of your project?

Short: There is no predictive skill, the test windows are short, and the data are free daily public data only.

Detail: Also: front-month futures without roll adjustment, small tuning grids, and a test window that was seen in earlier project versions.

Q55. Was your test set really unseen?

Short: The final pipeline reads it once and selects nothing on it; however earlier project versions were evaluated on the same window.

Detail: Be honest: the leak fix was triggered by an anomalous test result. The final decisions (parameters, members, weights) use validation only.

Q56. Could different dates change your conclusion?

Short: The conclusion is robust across all three assets and all model families, but the windows are short so individual numbers would move.

Detail: All six ML models are within about 0.5 % of the baseline on walk-forward folds of 582 (Bitcoin) / 396 (metals) samples each, which supports “no signal” beyond the test window.

Q57. Why are there negative R^2 and “worse than the mean” values?

Short: A negative R^2 means the model is worse than predicting the average; for noise-dominated targets any fitted model can be slightly worse out of sample.

Detail: Walk-forward R^2 of all models is within $-0.025 \dots +0.002$; the random walk itself has $R^2 \approx -0.005$.

Q58. Why do train and validation errors differ so much for the metals?

Short: The validation period was much more volatile; the random-walk baseline shows the same jump.

Detail: Gold: Ridge train 0.0097 vs validation 0.0203; Silver 0.019 vs 0.055. Naive validation RMSE: Silver 0.0554. It is a regime difference, not a model failure.

31.10 Future work questions

Q59. What would you improve with more time?

Short: Add richer information (news/sentiment, intraday data), adjust futures for contract rolls, and forecast volatility instead of direction.

Detail: Also multiple-testing control and a rolling forward test log.

Q60. Would more historical data help?

Short: Probably not much for this horizon: experiment E1 shows changes of about $-1.7 \% \dots +0.7 \%$ in validation RMSE when going from 3 years to the full history.

Detail: The project already moved from 3 years to 8+ years; the conclusion did not change.

Q61. Could you predict something that is more predictable?

Short: Yes — volatility. EWMA beat the persistence forecast by 4.4–7.7 % in log-volatility RMSE in experiment E4 and the conditional band already uses it.

Detail: Ridge/CatBoost improve on persistence for Bitcoin and Gold but not Silver. Volatility clustering is a well-known stable effect.

Q62. Why not use sentiment analysis of news?

Short: It needs extra data sources and careful timestamp alignment; we only use the Fear & Greed index for Bitcoin.

Detail: It would be a natural extension, but our feature-ablation shows the existing extra features add nothing at a 1-day horizon.

Q63. How would you deploy this?

Short: Containerise the Streamlit/FastAPI services, schedule data refresh, and monitor forward performance.

Detail: Docker configuration was previously removed from the repository (commit history), so this is future work, not current functionality.

Total: 63 questions.

32. “Explain My Project in 2 Minutes”

(≈ 290 words — read it aloud once to check the timing; use your own words.)

“Our project asks a simple question: can machine learning predict tomorrow’s price of Bitcoin, Gold and Silver better than just assuming tomorrow equals today? We downloaded daily prices from Yahoo Finance since 2018, plus the S&P 500, VIX, dollar index, oil, bond yields and the Bitcoin Fear and Greed index. From these we built about 28 or 29 features per day — recent returns, volatility, momentum indicators like RSI and MACD, and macro changes. We were very careful about time: gold and silver futures close at 1:30 pm New York time, so for them we only use macro data from the previous day; otherwise future information leaks in. We split the data by date — training until September 2025, validation until February 2026, and the rest as an unseen test period — and we tuned everything with walk-forward validation. We trained six models: Ridge, Random Forest, LightGBM, CatBoost, GRU and LSTM, and our final forecast is simply their average. The dashboard, built with Streamlit, lets the user pick an asset, press Run prediction and get tomorrow’s predicted price, UP or DOWN, and an uncertainty band — shown next to how reliable the forecast has been. **Our honest result is that no model beats the simple baseline.** The error is within half a percent of the baseline and direction accuracy is about fifty percent, which is what market theory predicts. What we can claim is a leak-free, reproducible pipeline that proves this, and an uncertainty band that covers about 74 to 76 percent of days. The limits are free daily data only, short test periods, and no news. It is a research prototype, not a trading system.”

33. “Explain My Project in 5 Minutes”

(≈ 760 words; the headings are for you, do not read them out.)

Introduction (30 s)

Good morning. Our project is a next-day price forecasting system for Bitcoin, Gold and Silver, with a Streamlit dashboard. But the real aim is a research question: can machine learning beat the simplest forecast — tomorrow equals today — and how can we test that honestly?

Problem statement (30 s)

Prices are very hard to predict. Daily returns have almost no autocorrelation — about minus 0.05 to minus 0.02 on our data — they have fat tails, and the price level itself is non-stationary. Many projects report high accuracy because they predict price levels or because information from the future leaks into the features. We wanted a pipeline that avoids both problems.

Dataset (40 s)

We use daily data from Yahoo Finance from January 2018 to September 2026: about 3,100 days for Bitcoin and 2,150 for Gold and Silver, plus S&P 500, VIX, dollar index, WTI oil, ten-year yield and the Fear and Greed index. We drop the still-running bar of the current day so every row is a finished day. We kept the downloaded files frozen, so every result is reproducible.

Preprocessing and features (50 s)

We predict the next-day log return, not the price. We built 28 to 29 stationary features: lagged returns, a 20-day momentum, rolling and EWMA volatility, RSI, ADX, MACD and Bollinger ratios, and macro returns. The key idea is the close-time rule: Bitcoin closes after the US markets so it may use same-day macro data; gold and silver settle at 1:30 pm New York time, so they only use the previous day's macro data and lagged high-low features. An earlier version without this rule showed 62 percent accuracy for silver — an artefact — and the audit removed it. The scaler is fitted only on training data.

Validation and models (60 s)

We split by date: training until September 2025, validation until February 2026, then an unseen test period of 207 days for Bitcoin and 143 for the metals. Hyper-parameters are chosen by four-fold expanding-window walk-forward validation. We trained Ridge, Random Forest, LightGBM, CatBoost, GRU and LSTM, compared against a random-walk, a historical-mean and an ARIMA baseline. The models use strong regularisation and early stopping.

Ensemble and prediction (40 s)

The served forecast is the equal-weight average of the six predicted returns, so nothing is selected on the test set. The predicted price is today's close times e to the power of the predicted return; it is UP if the return is positive. We also show a one-sigma band using EWMA volatility.

Evaluation and results (60 s)

We evaluate with return-space RMSE, R squared, direction accuracy with a binomial test and a Diebold-Mariano test against the random walk. On the unseen test set the combined forecast is within about half a percent of the random walk; R squared is about zero; direction accuracy is 46.9, 50.3 and 53.8 percent, none significant. So the honest conclusion is that next-day returns of these assets are not predictable with this data. The band, however, covers 74 to 76 percent of days against 68 percent nominal.

Streamlit demonstration (30 s)

In the dashboard the user selects an asset, presses Run prediction and sees the combined forecast, each model's own prediction, the band and the reliability table. A Predict-a-Day tab lets us replay any unseen day and reveal the actual result, and the performance tab shows all models against the baseline.

Conclusion (20 s)

We delivered a leakage-audited, reproducible pipeline, an honest negative result and a calibrated uncertainty band. Limitations are daily free data, short test windows and no news; future work is volatility forecasting, richer data and a forward test log. Thank you.

34. Presentation Slide Content

Sixteen slides for a ~15-minute talk. Images refer to files that already exist in the repository (paths given).

1. Title

Bullet points	Image / diagram	What the presenter says
- Multi-Asset Next-Day Return Prediction — Bitcoin, Gold, Silver - Team names, supervisor, University of Lahore (BSCS) - Tagline: a leakage-audited, walk-forward-validated comparison against the random walk	Dashboard screenshot (take a fresh one: Forecast tab after Run prediction).	Introduce the team and the question we asked, not “we built a predictor”.

2. Problem statement

Bullet points	Image / diagram	What the presenter says
- Next-day prices are hard: returns are noisy, fat-tailed (kurtosis 3.6–15), almost no autocorrelation ($\rho_1 \approx 0.02$–0.05) - Price levels are non-stationary (ADF $p \approx 1$) - Many projects over-report accuracy through leakage	results/figures/eda_return_distributions.png	Explain why a naive “tomorrow = today” forecast is a hard benchmark.

3. Objectives

Bullet points	Image / diagram	What the presenter says
- Build an end-to-end pipeline for 3 assets - Compare 6 ML models + baselines fairly - Prevent leakage; evaluate on an unseen test period - Serve a combined forecast in Streamlit	Simple 4-box list.	Primary: rigorous evaluation. Secondary: experiments, band, API.

4. Proposed solution

Bullet points	Image / diagram	What the presenter says
- Daily Yahoo Finance data 2018–2026 + macro + sentiment - Predict next-day log return → price = $P \cdot \exp(\hat{r})$ - Six models averaged equally - Honest reliability shown with every forecast	Pipeline diagram (Chapter 35).	One sentence per bullet.

5. System architecture

Bullet points	Image / diagram	What the presenter says
- data → features → split → tune/train → single test evaluation → inference → Streamlit/API - config.py single source; build_dataset single loader; backtesting single test reader	Chapter 35 diagram.	Walk left to right; point out the “only one place touches the test set”.

6. Dataset

Bullet points	Image / diagram	What the presenter says
- BTC-USD 3,177 rows; GC=F, SI=F 2,186 rows each - Daily OHLCV 2018-01-01 → 2026-09-11/12 - Macro: S&P 500, VIX, DXY, WTI, 10-y yield; Fear & Greed - Complete-bar rule; 0 nulls, 0 duplicates	results/figures/price_history.png	Show the split shading on the chart.

7. Features

Bullet points	Image / diagram	What the presenter says
28–29 stationary features - Returns/lags, momentum, volatility (HAR, EWMA), RSI/ADX/MACD/Bollinger ratios, macro returns - Close-time rule: same-day macro for Bitcoin, previous-day for the metals	Table from Chapter 7.2 (shortened) or a diagram of the two close times.	Stress the close-time rule and the Silver 62 % story as proof the audit works.

8. Machine-learning models

Bullet points	Image / diagram	What the presenter says
- Ridge · Random Forest · LightGBM · CatBoost · GRU · LSTM - Baselines: random walk, mean, ARIMA - Combined = mean of six predicted returns - Strong regularisation + early stopping	Table of Chapter 10 / hyper-parameters.	Explain each family in one line.

9. Training pipeline

Bullet points	Image / diagram	What the presenter says
- Frozen split: train $\leq$ 2025-09-10, val $\leq$ 2026-02-17, test after 4-fold expanding-window walk-forward tuning on train+val - Phase A (monitor val) $\rightarrow$ Phase B (refit train+val) - Scaler fitted on train only	Walk-forward diagram (Chapter 9).	Say “no shuffling, test touched once”.

10. Evaluation

Bullet points	Image / diagram	What the presenter says
- Metrics: RMSE/MAE/R² (returns), MAE/RMSE/MAPE (\$), direction accuracy + binomial test, Diebold-Mariano vs random walk - Everything compared with the random walk on the same days	results/figures/model_comparison_test.png	Explain DM test in one sentence.

11. Prediction workflow

Bullet points	Image / diagram	What the presenter says
- Refresh data $\rightarrow$ scale last 30 days $\rightarrow$ 6 models $\rightarrow$ average $\rightarrow$ $P \cdot \exp(\hat{r})$ - UP if $\hat{r} > 0$; band $P \cdot \exp(\hat{r} \pm \sigma_t)$ - Graceful failure when Yahoo/model/features are missing	Chapter 17 flow list.	Mention it runs in seconds.

12. Streamlit dashboard

Bullet points	Image / diagram	What the presenter says
- Sidebar: asset, Run prediction, overlays, Sync - Tabs: Forecast · Predict a Day · Test-Set History · Model Performance · Methodology - Reliability and disclaimer next to every forecast	Live demo or screenshots of each tab.	Demo Predict a Day: pick an unseen date, reveal the actual.

13. Results

Bullet points	Image / diagram	What the presenter says
- Combined vs random walk (test): RMSE within ± 0.4 %; DM p 0.49 / 0.97 / 0.46 - Direction 46.9 / 50.3 / 53.8 % (n.s.) - R² $\approx$ 0 for every model $\pm 1\sigma$ band coverage 74–76 % (nominal 68 %)	Table from Chapter 14.1 and results/figures/btc_actual_vs_predicted.png	Say the headline honestly: no model beats the baseline.

14. Limitations

Bullet points	Image / diagram	What the presenter says
- No predictive skill; short test windows (143–207 days) - Free daily data; futures roll not adjusted; no news - Test window seen in earlier project versions	Bulleted list only.	Examiners appreciate honesty; do not oversell.

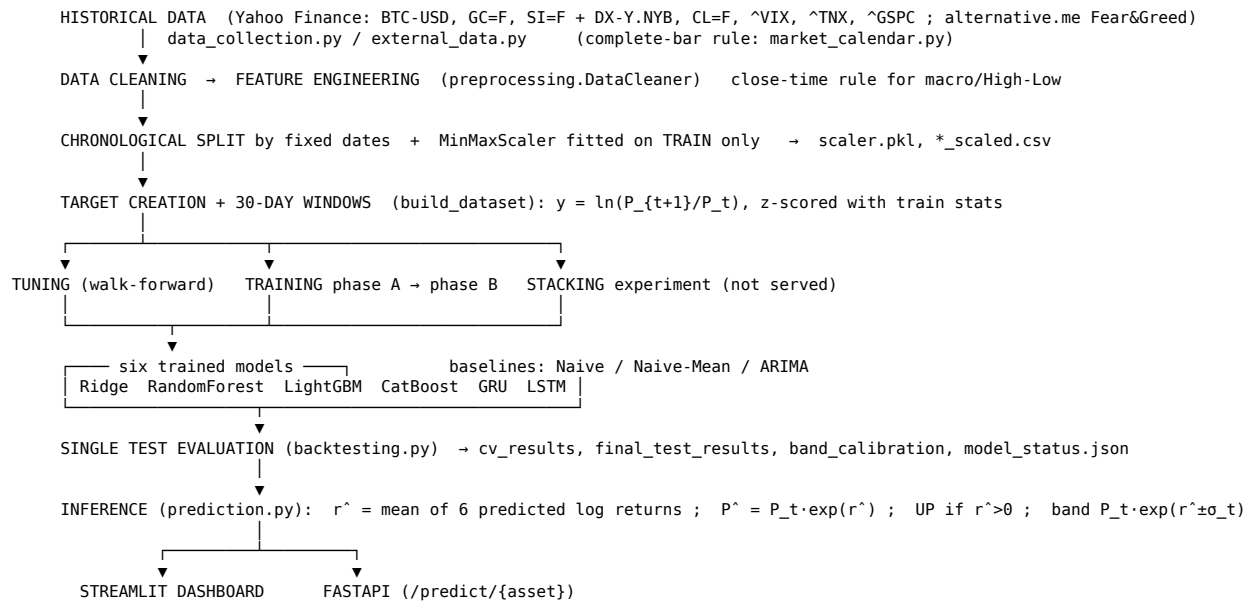
15. Future work

Bullet points	Image / diagram	What the presenter says
- Forecast volatility (EWMA/Ridge beat persistence in E4) - News/sentiment and intraday data; roll-adjusted futures - Forward test log; multiple-testing control	None.	Connect each item to a limitation.

16. Conclusion

Bullet points	Image / diagram	What the presenter says
- Leakage-audited, reproducible pipeline - Honest negative result consistent with market efficiency - Calibrated uncertainty band + transparent dashboard - Q&A	Single summary image or dashboard screenshot.	End with the one-sentence safe claim from Chapter 30.

35. System Architecture Diagram Explanation



Differences from the “textbook” diagram in the brief: (1) the Combined forecast is an unweighted average, so there is no model-selection step on the serving path; (2) a stacked ensemble exists only as an experiment; (3) a separate live-data refresh path (sync_live_data.py) feeds the dashboard without touching the frozen training data; (4) baselines are evaluated alongside the models in the same single test evaluation.

36. Important Terminology

Term	Plain-English meaning in our project
Machine learning	Computers learn a rule from examples (past days) instead of being programmed with the rule.
Supervised learning	Learning from examples that have the right answer attached (features of day t + the real return of day $t+1$).
Regression	Predicting a number (the return) rather than a category.
Classification	Predicting a category (e.g. UP/DOWN). We do not train a classifier; UP/DOWN is the sign of the regression output.
Time series	Data ordered in time where order matters.
Feature	An input number describing the market on a day (e.g. RSI, yesterday's return).
Target	The number we predict: tomorrow's log return.
Training / validation / test data	Past data to learn from / data to tune and stop early / data kept aside for the final one-time exam.
Overfitting	Memorising noise: great on training data, poor on new data.
Underfitting	Too simple to capture real patterns.
Data leakage	Information from the future or the test set sneaking into the inputs, inflating results.
Feature engineering	Turning raw prices into informative inputs (ratios, volatilities, indicators).
Hyper-parameter	A setting chosen before training (tree depth, learning rate, α).
Ensemble	Combining several models' predictions (here: simple average).
Prediction	The model's output for a new day.
Log return	$\ln(P_{t+1}/P_t)$: the percentage change on a log scale; $\approx$ the usual % change for small moves.
Random walk / naive forecast	"Tomorrow = today": predicted return 0.
Stationary	A series whose statistics do not drift over time (returns are, prices are not).
Walk-forward validation	Repeatedly train on the past and test on the next block, moving forward.
Volatility	How much returns fluctuate (standard deviation).
EWMA	Exponentially weighted moving average — recent days count more ($\lambda = 0.94$ here).
MAE	Mean absolute error — average size of a miss (lower is better).
RMSE	Root mean squared error — like MAE but punishes big misses (lower is better).
MAPE	Mean absolute percentage error of the price; misleading here because the naive forecast scores the same.
R^2	$1 - \text{error}/\text{variance}$; 0 = no better than predicting the mean, negative = worse.
Directional accuracy	Share of days where the predicted sign matches the actual sign (50 % = coin flip).
Diebold–Mariano test	Statistical test of whether two forecasts differ in accuracy.
p-value	Probability of seeing a result at least this extreme if there were no real effect; < 0.05 is conventionally "significant".
Scaling / Normalisation	Rescaling features to a common range (MinMax: 0–1).
Standardisation	Subtracting the mean and dividing by the standard deviation (z-score). We use it for the target.
API	A way for programs to talk to each other: Yahoo/alternative.me provide data APIs; our FastAPI offers a prediction API.
Streamlit	Python library that turns a script into a web app.
Model serialization	Saving a trained model to a file (.pkl, .cbm, .keras) and loading it later.
Early stopping	Stop training when validation error stops improving.
Complete-bar rule	Never use today's unfinished daily row.
Close-time rule	A feature may use only data available when the asset's daily close is fixed.

37. Code Explanation for Presentation

The code sections an examiner is most likely to ask about while you share your screen. Snippets are quoted from the repository.

37.1 Close-time-safe macro alignment

src/data/preprocessing.py — DataCleaner._align_external

```
if EXTERNAL_SAME_DAY[self.config['type']]:
    return series.reindex(self.df.index, method='ffill')
pos = series.index.searchsorted(self.df.index, side='left') - 1      # last external date < t
vals = np.where(pos >= 0, series.values[np.clip(pos, 0, None)], np.nan)
return pd.Series(vals, index=self.df.index)
```

What it does	For Bitcoin uses the same-day external value; for the futures uses the last external value strictly before day t.
Why it is needed	Gold/Silver settle at 13:30 ET before the S&P/VIX/DXY closes; same-day values would contain the future.
If removed	Silver's apparent 62 % accuracy returns (a leak); results become meaningless.
Input	external daily series (return series), asset calendar
Output	aligned series
Called by	add_external_features

37.2 Train-only scaling and chronological split

src/data/preprocessing.py — DataCleaner.normalize_and_split

```
train_df = self.df.loc[:TRAIN_END, cols]
val_df   = self.df.loc[pd.Timestamp(TRAIN_END) + pd.Timedelta(days=1):VAL_END, cols]
test_df  = self.df.loc[pd.Timestamp(VAL_END) + pd.Timedelta(days=1):, cols]
self.scaler = MinMaxScaler(feature_range=(0, 1)).fit(train_df)
```

What it does	Splits by fixed dates and fits the scaler on the training rows only.
Why it is needed	Prevents the test/validation distribution from influencing the scaling.
If removed	If fitted on all data the test statistics would leak into training inputs.
Input	cleaned feature frame
Output	scaled train/val/test frames, saved scaler
Called by	run_cleaning_pipeline

37.3 Target creation and date-based masks

src/data/preprocessing.py — _task_target and build_dataset

```
fwd = sum(r.shift(-k) for k in range(1, h + 1))      # h = 1 → tomorrow's log return
...
masks = {'train': (last_target <= t_end) & ...,
         'val':   (first_target > t_end) & (last_target <= v_end),
         'test':  first_target > v_end}
```

What it does	Creates the next-day target and assigns each sample to a split by the dates its <i>target</i> covers.
Why it is needed	Defines what the model predicts and guarantees train targets never cross into validation/test.
If removed	Without the shift there is no prediction problem; without target-date masks multi-day targets would overlap splits.
Input	features CSV, scaled CSVs
Output	dict of windows, targets, dates, prices
Called by	tune/train/ensemble/backtesting/inference tests

37.4 Walk-forward evaluation

src/evaluation/cross_validation.py — cv_evaluate

```
for k, (tr, va) in enumerate(walk_forward_splits(len(y), n_splits)):
    n_inner = int(len(tr) * inner_val_frac)
    tr_fit, tr_mon = tr[:-n_inner], tr[-n_inner:]
    m = make_model(model_name, params)
    m.fit(Xs[tr_fit], Xt[tr_fit], y[tr_fit], Xs[tr_mon], Xt[tr_mon], y[tr_mon], final=True)
    pred = m.predict(Xs[va], Xt[va])
```

What it does	Trains on the past of each fold (monitoring on its last 15 %) and predicts the fold's block.
Why it is needed	Gives several out-of-sample estimates for tuning and model comparison without the test set.
If removed	We would have to tune on the test set or a single small validation window.
Input	model name, params, dataset dict
Output	per-fold metrics
Called by	tune_models, backtesting, ensemble_model, run_experiments

37.5 Two-phase LightGBM fit

src/models/registry.py — LightGBMModel.fit

```
m1 = lgb.LGBMRegressor(n_estimators=n_est, **base)
m1.fit(Xt_tr, y, eval_set=[(Xt_val, y_val)], callbacks=[lgb.early_stopping(30, verbose=False)])
best_n = max(10, int(m1.best_iteration_ or n_est))
...
self.model = lgb.LGBMRegressor(n_estimators=best_n, **base).fit(X_train_plus_val, y)
```

What it does	Finds the number of trees on validation, then refits on train+val with that number.
Why it is needed	Avoids reporting a validation score on data that also chose the stopping point.
If removed	Optimistic validation metrics, or a deployed model that never used the validation data.
Input	train/val arrays
Output	fitted model
Called by	train_asset, cv_evaluate

37.6 The combined prediction

src/inference/prediction.py — _predict_from_window, predict_next_day

```
members = {m: _predict_one(asset, m, Xseq, Xt) for m in combined_members(asset)}
return float(np.mean(list(members.values()))), members
...
pred_price = last_close * np.exp(r_hat)
'direction': 'UP' if r_hat > 0 else 'DOWN',
```

What it does	Every trained model predicts a log return; the mean is converted to a price and UP/DOWN.
Why it is needed	It is the actual forecast served to the user.
If removed	The dashboard would have no prediction.
Input	scaled window, saved models
Output	$\hat{r}$, member predictions, price, direction
Called by	predict_next_day, predict_for_date, backtesting (mirror logic)

37.7 Single test-set evaluation

src/evaluation/backtesting.py — test_predictions / main

```
members = [m for m in ML_MODELS if m in preds]
preds[COMBINED] = np.mean([preds[m] for m in members], axis=0)
...
best = ml.sort_values('RMSE_ret_mean').iloc[0] # model selection uses walk-forward (validation) results only
```

What it does	Predicts the test split once for every model and the Combined forecast, writes all result tables and model_status.json.
Why it is needed	It is the single, final, honest evaluation.
If removed	No results, no dashboard metrics, no model_status.json (target mean/std needed for prediction).
Input	saved models, test arrays
Output	CSV/JSON/MD results
Called by	Makefile evaluate

37.8 Conditional uncertainty band

src/inference/prediction.py — _band

```
sigma = float(feats.loc[as_of_ts, BAND_SIGMA_FEATURE]) # ewma_vol at day t
return {'uncertainty_band': [float(last_close * np.exp(r_hat - sigma)), float(last_close * np.exp(r_hat + sigma))], ...}
```

What it does	Builds the $\pm 1\sigma$ price band using the EWMA volatility known on day t.
--------------	---

Why it is needed	A calibrated indication of how wrong a forecast typically is.
If removed	The forecast would be a bare number; the earlier fixed band was mis-calibrated (44–87 % validation coverage).
Input	features, close, $\hat{r}$
Output	band and σ in %
Called by	<code>predict_next_day</code> , <code>predict_for_date</code>

37.9 Diebold-Mariano test

`src/utils/metrics.py` — `diebold_mariano`

```
d = (true_ret - pred_a) ** 2 - (true_ret - pred_b) ** 2
dm = d_bar / np.sqrt(var / n)
dm *= np.sqrt((n + 1 - 2 * h + h * (h - 1) / n) / n)      # Harvey–Leybourne–Newbold
p = 2 * (1 - stats.t.cdf(abs(dm), df=n - 1))
```

What it does	Tests whether a model's squared errors differ significantly from the baseline's.
Why it is needed	Without it a 0.3 % RMSE gap would be reported as improvement.
If removed	We could not distinguish luck from skill.
Input	true returns, two predictions
Output	(statistic, p-value)
Called by	<code>evaluate_forecast</code>

37.10 Run-prediction logic in the dashboard

`app/streamlit_app.py` — Forecast tab

```
if not (live_data_is_current(asset) or fresh):          # download at most once per 30 min while the vendor lags
    refreshed = update_live_data(asset, refresh_external=True)
...
r = predict_next_day(asset, COMBINED)
```

What it does	Refreshes data only when stale, then calls the single prediction function.
Why it is needed	Keeps the UI thin and avoids hammering Yahoo.
If removed	The UI would show stale data or crash on network errors.
Input	asset selection, session state
Output	prediction result dict rendered as cards/tables/charts
Called by	Streamlit runtime

37.11 Lagging the post-settlement features (futures)

`src/data/preprocessing.py` — `lag_post_settlement_features`

```
for c in POST_SETTLEMENT_FEATURES:                    # hl_range, atr_norm, ADX
    self.df[c] = self.df[c].shift(1)
```

What it does	Shifts High/Low-based features by one session for the metals.
Why it is needed	Yahoo's daily High/Low cover trading after the 13:30 ET settlement, i.e. inside the target interval.
If removed	Part of the leakage that produced the fake Silver result would come back.
Input	feature frame
Output	lagged columns
Called by	<code>clean_data</code> (commodities only)

38. Actual Results

All numbers are read directly from `results/final_test_results.csv` and `results/cv_results.csv` when this PDF was generated. Definitions: **Test** = the untouched split (target dates 2026-02-18 → 2026-09-12 for Bitcoin, 207 days; 2026-02-18 → 2026-09-11 for Gold and Silver, 143 days), evaluated once by `backtesting.py`. **Walk-forward (validation)** = mean over 4 expanding-window folds inside train+val. RMSE/MAE/R² are in return space; \$ metrics are in USD; DA = directional accuracy on non-flat days (binomial p-value in brackets); UP calls = share of days predicted UP; DM p = Diebold–Mariano p-value vs the random walk. Models marked **Combined** are the served forecast; Stacked is the experiment.

38.1 Bitcoin

Test set (207 days). Baseline = Naive (random walk).

Model	MAE (\$)	RMSE (\$)	MAPE %	RMSE (ret)	R ² (ret)	Dir. acc % (p)	UP calls %	DM p	Strategy %	Buy&Hold %
Combined	1,063.94	1,447.37	1.52	0.02072	+0.006	46.9 (0.83)	37	0.49	-5.1	+14.5
Naive	1,066.42	1,451.73	1.52	0.02079	-0.001	—	—	—	+0.0	+14.5
Naive-Mean	1,067.32	1,451.38	1.53	0.02078	-0.000	49.3 (0.61)	100	0.81	+14.4	+14.5
ARIMA	1,067.96	1,459.44	1.53	0.02091	-0.012	46.9 (0.83)	72	0.35	-9.0	+14.5
Ridge	1,071.97	1,452.59	1.53	0.02080	-0.002	47.8 (0.76)	39	0.91	+4.4	+14.5
RandomForest	1,064.48	1,448.81	1.52	0.02075	+0.004	48.3 (0.71)	57	0.62	-3.6	+14.5
LightGBM	1,076.49	1,446.18	1.54	0.02068	+0.010	51.2 (0.39)	45	0.79	+5.4	+14.5
CatBoost	1,071.07	1,453.76	1.53	0.02082	-0.004	46.9 (0.83)	60	0.74	-5.6	+14.5
GRU	1,075.86	1,458.31	1.54	0.02089	-0.010	46.9 (0.83)	16	0.35	-14.0	+14.5
LSTM	1,068.06	1,453.44	1.53	0.02082	-0.004	46.4 (0.87)	26	0.35	-7.8	+14.5
Stacked	1,061.35	1,442.94	1.52	0.02065	+0.013	50.2 (0.50)	64	0.44	+2.3	+14.5

Walk-forward validation (4 folds, train+val).

Model	RMSE (ret) mean ± std	MAE (ret)	R ² (ret)	Dir. acc %	RMSE (\$)
CatBoost	0.03156 ± 0.00695	0.02136	-0.001	50.2	1,427.07
LightGBM	0.03158 ± 0.00692	0.02140	-0.003	51.2	1,430.52
Combined	0.03159 ± 0.00693	0.02133	-0.004	50.8	1,429.51
Naive	0.03160 ± 0.00690	0.02130	-0.005	—	1,429.08
RandomForest	0.03160 ± 0.00688	0.02140	-0.005	50.0	1,432.90
Ridge	0.03161 ± 0.00693	0.02135	-0.005	49.4	1,430.34
Naive-Mean	0.03163 ± 0.00694	0.02133	-0.007	48.8	1,430.95
ARIMA	0.03164 ± 0.00695	0.02135	-0.007	48.1	1,431.80
GRU	0.03168 ± 0.00697	0.02136	-0.009	49.4	1,432.61
LSTM	0.03170 ± 0.00697	0.02142	-0.010	48.8	1,433.09

38.2 Gold

Test set (143 days). Baseline = Naive (random walk).

Model	MAE (\$)	RMSE (\$)	MAPE %	RMSE (ret)	R ² (ret)	Dir. acc % (p)	UP calls %	DM p	Strategy %	Buy&Hold %
Combined	58.20	75.73	1.29	0.01671	-0.001	50.3 (0.50)	97	0.97	-7.9	-10.6
Naive	58.26	75.76	1.29	0.01671	-0.002	—	—	—	+0.0	-10.6
Naive-Mean	58.39	75.99	1.30	0.01675	-0.007	49.7 (0.57)	100	0.43	-10.7	-10.6
ARIMA	58.57	76.27	1.30	0.01681	-0.015	50.3 (0.50)	80	0.52	-5.8	-10.6
Ridge	58.51	76.05	1.30	0.01677	-0.009	50.3 (0.50)	94	0.76	-7.6	-10.6
RandomForest	58.08	75.10	1.29	0.01659	+0.012	49.0 (0.63)	87	0.57	-10.4	-10.6
LightGBM	58.25	75.81	1.29	0.01672	-0.003	49.7 (0.57)	100	0.88	-10.7	-10.6
CatBoost	58.43	76.22	1.30	0.01681	-0.015	51.7 (0.37)	92	0.71	-7.0	-10.6
GRU	58.00	75.42	1.29	0.01664	+0.007	51.7 (0.37)	97	0.58	-6.5	-10.6
LSTM	58.82	76.58	1.31	0.01687	-0.022	49.7 (0.57)	100	0.28	-10.7	-10.6
Stacked	58.95	77.28	1.31	0.01705	-0.043	51.0 (0.43)	97	0.44	-7.9	-10.6

Walk-forward validation (4 folds, train+val).

Model	RMSE (ret) mean $\pm$ std	MAE (ret)	R ² (ret)	Dir. acc %	RMSE (\$)
CatBoost	0.01110 $\pm$ 0.00219	0.00786	-0.001	54.8	28.29
Ridge	0.01112 $\pm$ 0.00223	0.00787	-0.003	54.8	28.40
Combined	0.01113 $\pm$ 0.00222	0.00788	-0.004	52.5	28.39
Naive-Mean	0.01113 $\pm$ 0.00223	0.00787	-0.004	55.8	28.43
LightGBM	0.01113 $\pm$ 0.00223	0.00787	-0.004	55.8	28.42
GRU	0.01114 $\pm$ 0.00222	0.00791	-0.007	51.1	28.45
Naive	0.01114 $\pm$ 0.00225	0.00790	-0.006	—	28.46
LSTM	0.01116 $\pm$ 0.00223	0.00793	-0.010	47.0	28.49
RandomForest	0.01116 $\pm$ 0.00220	0.00791	-0.012	51.9	28.45
ARIMA	0.01123 $\pm$ 0.00219	0.00796	-0.025	50.5	28.64

38.3 Silver

Test set (143 days). Baseline = Naive (random walk).

Model	MAE (\$)	RMSE (\$)	MAPE %	RMSE (ret)	R ² (ret)	Dir. acc % (p)	UP calls %	DM p	Strategy %	Buy&Hold %
Combined	1.78	2.37	2.47	0.03180	+0.005	53.8 (0.20)	91	0.46	-6.0	-12.1
Naive	1.79	2.37	2.49	0.03189	-0.001	—	—	—	+0.0	-12.1
Naive-Mean	1.79	2.38	2.49	0.03192	-0.003	51.7 (0.37)	100	0.63	-12.2	-12.1
ARIMA	1.81	2.37	2.52	0.03226	-0.024	52.4 (0.31)	55	0.50	-3.2	-12.1
Ridge	1.79	2.37	2.48	0.03181	+0.005	54.5 (0.16)	94	0.48	-0.7	-12.1
RandomForest	1.76	2.33	2.44	0.03146	+0.026	56.6 (0.07)	66	0.17	+6.5	-12.1
LightGBM	1.79	2.37	2.48	0.03189	-0.001	58.7 (0.02)	86	0.98	+10.1	-12.1
CatBoost	1.79	2.38	2.49	0.03193	-0.003	51.7 (0.37)	100	0.70	-12.2	-12.1
GRU	1.79	2.37	2.48	0.03189	-0.001	55.2 (0.12)	88	0.99	+3.7	-12.1
LSTM	1.80	2.38	2.49	0.03198	-0.006	51.7 (0.37)	100	0.63	-12.2	-12.1
Stacked	1.79	2.38	2.48	0.03191	-0.002	52.4 (0.31)	94	0.91	-11.0	-12.1

Walk-forward validation (4 folds, train+val).

Model	RMSE (ret) mean $\pm$ std	MAE (ret)	R ² (ret)	Dir. acc %	RMSE (\$)
GRU	0.02387 $\pm$ 0.00625	0.01592	+0.002	52.3	1.00
LightGBM	0.02389 $\pm$ 0.00625	0.01592	-0.000	53.3	1.00
Combined	0.02390 $\pm$ 0.00624	0.01592	-0.001	52.7	1.00
CatBoost	0.02391 $\pm$ 0.00623	0.01594	-0.002	53.5	1.00
Naive-Mean	0.02391 $\pm$ 0.00623	0.01594	-0.002	53.5	1.00
Naive	0.02391 $\pm$ 0.00624	0.01596	-0.002	—	1.00
Ridge	0.02393 $\pm$ 0.00625	0.01593	-0.003	53.3	1.00
LSTM	0.02393 $\pm$ 0.00625	0.01595	-0.003	51.2	1.00
RandomForest	0.02396 $\pm$ 0.00619	0.01596	-0.007	51.1	1.01
ARIMA	0.02410 $\pm$ 0.00613	0.01619	-0.021	50.2	1.00

38.4 Uncertainty band ($\pm 1\sigma$, nominal coverage 68.3 %)

Asset	Split	Band	Days	Coverage %	QLIKE	Mean σ %
Bitcoin	validation	conditional	160	75.0	1.875	2.18
Bitcoin	validation	fixed	160	87.5	2.029	3.36
Bitcoin	test	conditional	207	73.9	1.767	2.21
Bitcoin	test	fixed	207	88.9	2.037	3.32
Gold	validation	conditional	109	72.5	2.039	1.57
Gold	validation	fixed	109	53.2	3.833	0.98
Gold	test	conditional	143	75.5	1.392	1.84
Gold	test	fixed	143	51.0	1.929	1.06
Silver	validation	conditional	109	57.8	1.936	3.70
Silver	validation	fixed	109	45.0	7.403	1.92

Asset	Split	Band	Days	Coverage %	QLIKE	Mean σ %
Silver	test	conditional	143	75.5	1.588	3.99
Silver	test	fixed	143	54.5	1.806	2.27

38.5 Experiment highlights (validation only, from results/experiments/)

- **E1 data size:** going from 3 years to the full history changed validation RMSE by -1.7% ... $+0.7\%$ — no consistent gain.
- **E2 feature ablation:** dropping any feature group changes walk-forward RMSE by -0.4% ... $+0.05\%$.
- **E3 horizon:** 5-day returns are not easier: RMSE vs naive ranges -1.0% ... $+1.4\%$ across models and assets.
- **E4 volatility (22-day log RV):** EWMA beats persistence by 7.7% (Bitcoin), 7.0% (Gold), 4.4% (Silver) in log-RMSE — volatility is more predictable than direction.
- **Regimes:** combined forecast $\approx$ random walk in low/medium/high-volatility and up/down-trend regimes (results/regime_analysis.csv). Day-type splits (4% hit on Gold down days, 97% on up days) merely reflect the always-UP behaviour.

Reading the results. No model has a significant edge over the random walk (smallest DM p over all models and assets: 0.17). The numbers vary by a few hundredths of a percent between models because the models predict almost the same near-zero return.

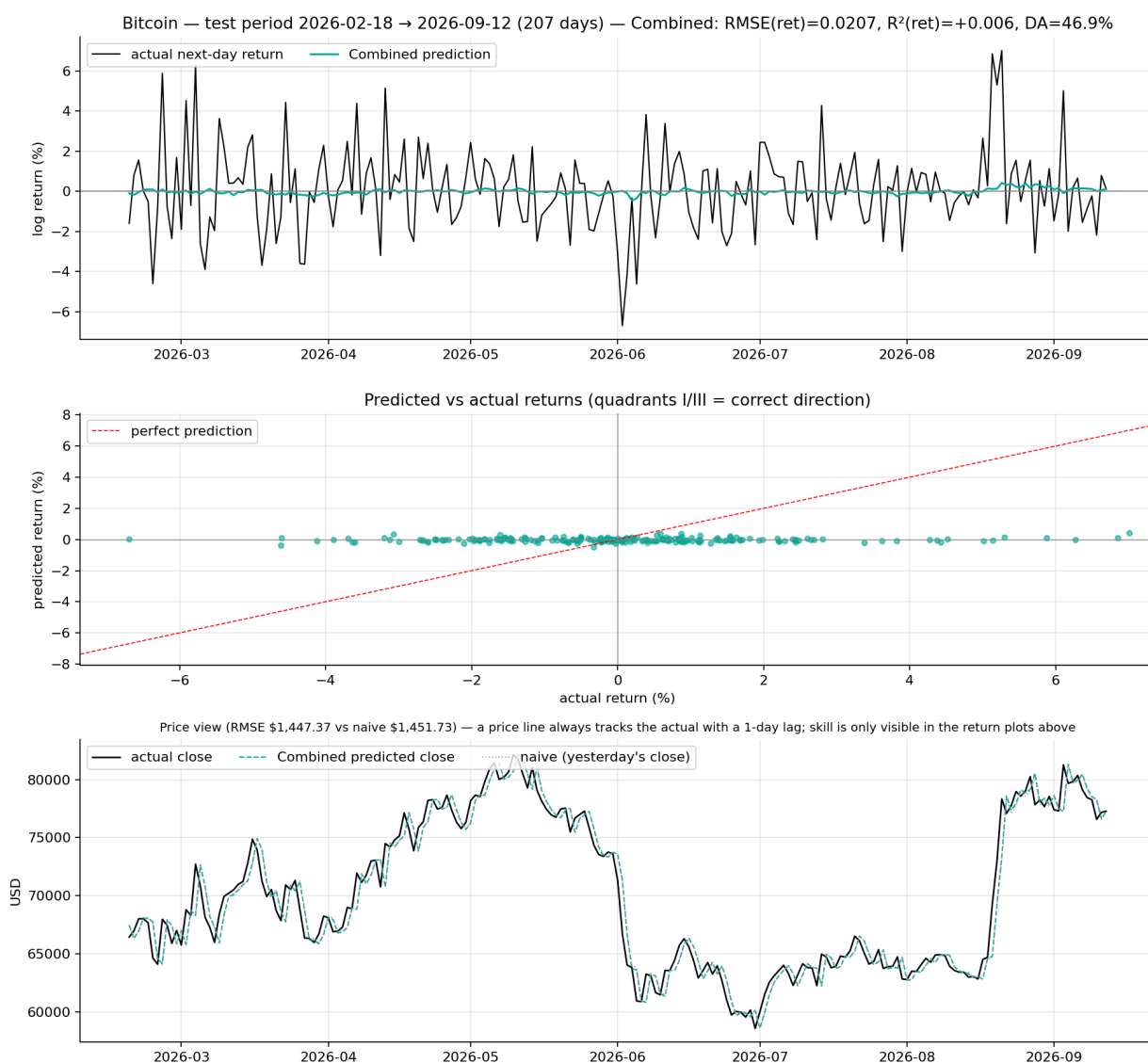


Figure 38.1 — Bitcoin, test period: actual vs predicted returns and prices (results/figures/btc_actual_vs_predicted.png).

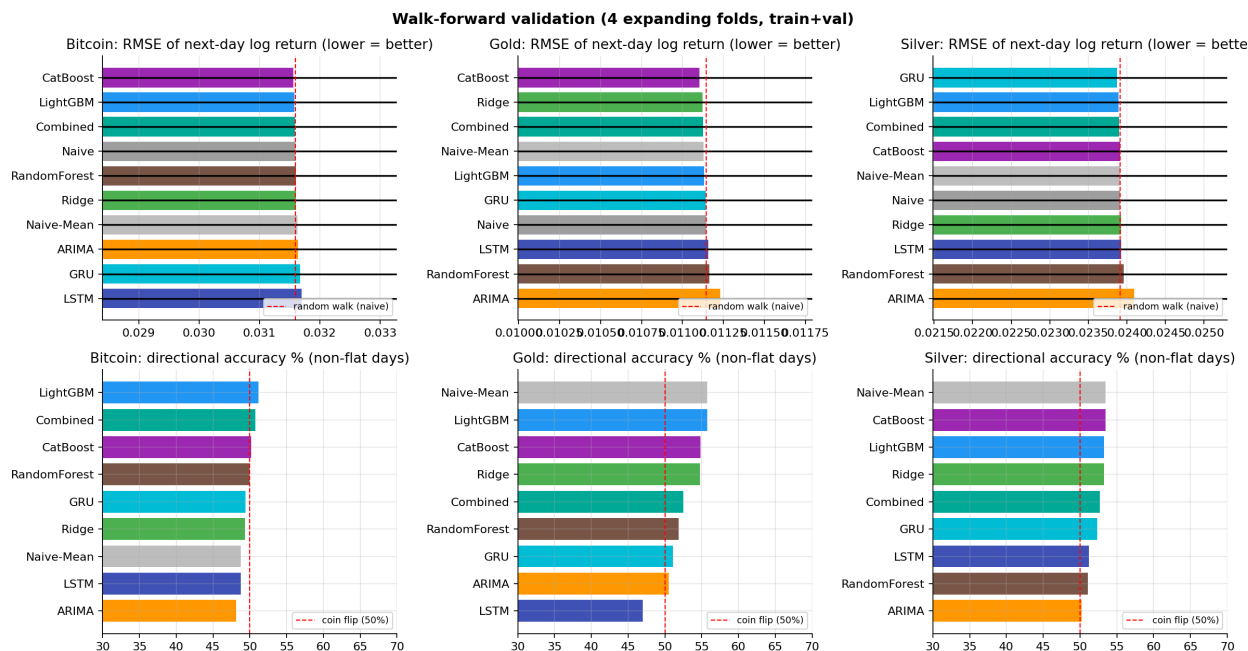


Figure 38.2 — Walk-forward validation comparison (results/figures/model_comparison_cv.png).

39. Honest Project Assessment

Question	Answer
Is the project technically working?	Yes. On this machine the 43 tests pass (62 s); the real prediction path produced forecasts for all three assets; all five dashboard tabs rendered for Bitcoin and Silver without exceptions in a headless run; the API code paths are covered by tests of the shared prediction functions.
Is the ML pipeline logically correct?	Yes. Causal features, train-only scaling, date-based split, walk-forward tuning on train+val, two-phase fitting, one test evaluation, a baseline in every table.
Is there data leakage?	Not in the final pipeline (unit tests + my truncation test). A real leak existed earlier and was fixed. Residual: Silver's same-day gold_return assumption; test window seen in earlier versions.
Are the evaluation results trustworthy?	As a "no skill" conclusion: yes. Metrics, significance tests and baseline are correct. As precise numbers they are noisy (143–207 test days).
Are the predictions meaningful enough for a university FYP?	The <i>engineering and methodology</i> are strong and defensible. The <i>forecasts</i> themselves carry no information beyond "tomorrow $\approx$ today". An FYP built on an honest negative result plus a calibrated band is defensible — if you present it that way. If an examiner expects "high accuracy", redirect to the baseline comparison.
What can we confidently claim?	A leakage-audited, reproducible pipeline; a fair comparison of six models and three baselines; that no model beats the random walk (DM $p \geq 0.17$); that the close-time rule matters (the Silver 62 % case); that the EWMA band is reasonably calibrated; a working dashboard and API.
What should we NOT claim?	That the system predicts prices or direction; that it can make money; that Silver/LightGBM (58.7 %, $p = 0.02$) is a discovery; that the ensemble "improves" accuracy; that the test set was never touched in the project's history; that more data or deeper models would fix it.
What questions could expose weaknesses?	"Why does every model stop after 1 epoch?", "Why does Gold say UP 97 % of the time?", "Your MAPE equals the naive MAPE", "Was the test set really unseen?", "Why average six models that have no skill?", "Why are Ridge α and RF depth at the grid edges?", "Futures rolls?", "How would a longer test window change things?"
What should we fix / prepare before the presentation?	Nothing blocks the demo. Recommended (low effort): (1) the stored live data ends mid-September — run "Sync live market data" while online the day before, and keep the offline "Predict a Day" tab as a fallback; (2) rehearse the honest-framing answers in Chapter 31; (3) decide whether to mention minor code notes (LightGBM subsample, silent skipping of missing members, unused requests, test-only create_sequences) — they do not change results; (4) CHANGELOG entries still mention older test counts (34/41) from earlier dates — the current count is 43; (5) take fresh dashboard screenshots for the slides.

Bottom line. You can stand behind this project. Its strength is that it does not fool itself. The best viva strategy is to say the negative result first, then show the evidence that it is real and not a bug.

40. Final “Student Cheat Sheet”

Topic	Say this
Our project in one sentence	A leak-free ML pipeline and Streamlit dashboard that forecasts next-day Bitcoin, Gold and Silver returns and honestly shows that no model beats “tomorrow = today”.
Our problem	Can ML predict next-day prices better than a naive forecast, tested without data leakage?
Our solution	Daily data → stationary features → chronological split → 6 models + baselines → one test evaluation → combined forecast + $\pm 1\sigma$ band in Streamlit.
Our dataset	Yahoo Finance daily OHLCV 2018-01-01 → 2026-09-12 (Bitcoin 3,177 rows) / 2026-09-11 (Gold, Silver 2,186 rows) + S&P 500, VIX, DXY, WTI, 10-y yield, Fear & Greed.
Our assets	BTC-USD (crypto, 24/7, closes 00:00 UTC); GC=F, SI=F (COMEX futures, settle 13:30 ET).
Our target	Next-day log return $\ln(P_{t+1}/P_t)$; price = $P_t \cdot \exp(\hat{r})$.
Our features	29 / 28 / 29 stationary features: returns + lags + 20-day momentum, volatility (rolling, HAR, EWMA), RSI, ADX, ROC, MACD/EMA/Bollinger ratios, ATR, macro returns, Fear & Greed (BTC), gold return (Silver).
Our models	Ridge, Random Forest, LightGBM, CatBoost, GRU, LSTM (+ baselines Naive, Naive-Mean, ARIMA; stacked experiment).
Our best-supported results	Combined vs random walk on the unseen test: RMSE(ret) 0.02072 vs 0.02079 (BTC), 0.01671 vs 0.01671 (Gold), 0.03180 vs 0.03189 (Silver); DM p 0.49 / 0.97 / 0.46; direction 46.9 / 50.3 / 53.8 %; $R^2 \approx 0$; band coverage 73.9 / 75.5 / 75.5 %.
Our ensemble	Equal-weight mean of the six predicted log returns; no fitted weights.
Our prediction flow	refresh data → scale last 30 days → 6 models → average → $P \cdot \exp(\hat{r})$ → UP if $\hat{r} > 0$ → $\pm 1\sigma$ band.
Our Streamlit application	Sidebar (asset, Run prediction, overlays, Sync) + tabs: Forecast, Predict a Day, Test-Set History, Model Performance, Methodology.
Our biggest limitation	No predictive skill beyond the random walk (data are free daily public data; short test windows).
Our biggest technical strength	Leakage-aware, time-series-correct evaluation (close-time rule, train-only scaler, walk-forward, single test read, tests).
Why machine learning	To test whether non-linear models can find patterns in market features that a naive forecast cannot.
Why multiple models	Different families, different mistakes; fair comparison; averaging reduces variance.
Why time-series splitting matters	Random splits leak neighbouring days; chronological splits mimic real forecasting.
Why data leakage is dangerous	It makes models look skilled (our Silver 62 % was fake) and fail in reality.
How our final prediction is produced	Each model predicts a standardised return → $\times \text{std} + \text{mean}$ → average of six → $P_t \cdot \exp(\hat{r})$.
What we would improve	Forecast volatility, add news/intraday data, roll-adjusted futures, forward test log, multiple-testing control.

Ten numbers to memorise

Number	Meaning
3,177 / 2,186	raw rows Bitcoin / Gold = Silver
29 / 28 / 29	features Bitcoin / Gold / Silver
30	days in the GRU/LSTM window (SEQ_LEN)
2025-09-10 / 2026-02-17	train end / validation end
207 / 143 / 143	test days Bitcoin / Gold / Silver
4	walk-forward folds
6	models in the Combined forecast
0.49 / 0.97 / 0.46	Diebold-Mariano p of Combined vs random walk
46.9 / 50.3 / 53.8 %	directional accuracy of Combined
43	tests passing

Appendix A — Verification Record

What was actually inspected and executed to write this guide (2026-10-03):

Check	Result
Source files read in full	config.py, Makefile, README, requirements, .env.example, CI workflow, preprocessing.py, data_collection.py, external_data.py, market_calendar.py, sync_live_data.py, eda.py, registry.py, model_gru.py, model_lstm.py, ensemble_model.py, tune_models.py, train_models.py, cross_validation.py, backtesting.py, prediction.py, api/app.py, metrics.py, inverse_transform.py, reproducibility.py, logging_config.py, retrain.py, run_experiments.py, before_after.py, streamlit_app.py; test and notebook structure; CHANGELOG; result CSV/JSON/MD files.
Data facts	Row counts, date ranges, nulls, duplicates, column names, split sizes read from data/raw, data/processed, data/models/model_status.json.
Test suite	python -m pytest tests -q → 43 passed in 62 s (Python 3.9.6; warnings only).
Look-ahead truncation test	Raw files cut at 2026-06-01, features rebuilt with DataCleaner, compared with the stored full-data features on the common dates: max difference 1.5e-11 (Bitcoin), 9.1e-13 (Gold), 3.1e-13 (Silver) → no look-ahead.
Scaler check	Training data scales to exactly [0, 1]; test cells outside [0, 1]: 0.02 % / 3.4 % / 2.6 % → scaler not fitted on test.
Real prediction path	predict_next_day for all assets and predict_for_date("Gold", "2026-05-12") executed; outputs quoted in Chapter 27.
Dashboard smoke test	Streamlit AppTest: five tabs rendered for Bitcoin and Silver, 0 exceptions.
Keras parameter counts	GRU 6,593 / 6,497; LSTM 8,481 / 8,353 loaded from data/models/*.keras.
Not done	The training pipeline was not re-run (no re-tuning/re-training; results are the stored ones); no network download was performed; the FastAPI server was not started (its logic is shared with the verified prediction functions).

Final consistency check. Numerical tables in Chapters 14, 15, 38 and 40 are generated from the CSV files by the PDF-building script; statements about code were checked against the source quoted. Items that are judgments rather than facts (e.g. “the project can be defended”) are labelled in Chapter 39.